A VERSION-AWARE RAG FOR RELIABLE NETWORK CLI CONFIGURATION SYNTHESIS

A PROJECT REPORT

Submitted to

Amrita Vishwa Vidyapeetham

in partial fulfilment for the award of the degree of

BACHELORS OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

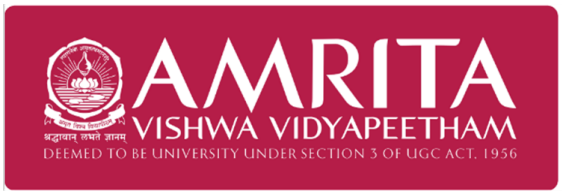
By

ARYAN MADHUSUDHANAN
(Reg. No. CH.SC.U4CSE23207)

SRIVISHNU T
(Reg. No. CH.SC.U4CSE23249)

Supervisor

Mrs. V Nivethitha



AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF COMPUTING
CHENNAI - 601103

April 2026

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

SCHOOL OF COMPUTING CHENNAI

BONAFIDE CERTIFICATE

This is to certify that this project report entitled **"A VERSION-AWARE RAG FOR RELIABLE NETWORK CLI CONFIGURATION SYNTHESIS"** is the bonafide work of **"Aryan Madhusudhanan (CH.SC.U4CSE23207) & Srivishnu T (CH.SC.U4CSE23249)"** who carried out the project work under my supervision.

SIGNATURE

DR. S. BAGHAVATHI PRIYA
CHAIRPERSON

Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Chennai - 601103

SIGNATURE

MRS. V NIVETHITHA
SUPERVISOR

Assistant Professor,
Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Chennai - 601103

INTERNAL EXAMINER

EXTERNAL EXAMINER

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

SCHOOL OF COMPUTING CHENNAI

DECLARATION BY THE CANDIDATE

We hereby declare that the project report entitled **"A VERSION-AWARE RAG FOR RELIABLE NETWORK CLI CONFIGURATION SYNTHESIS"** is a bona fide record of the Phase 1 work done by us under the guidance of our supervisor. This work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or other similar title to any candidate of any University.

SIGNATURE
Aryan Madhusudhanan
(CH.SC.U4CSE23207)

SIGNATURE
Srivishnu T
(CH.SC.U4CSE23249)

ABSTRACT

The combination of Large Language Models with Intent-Based Networking systems enables organizations to achieve their operational goals through remote operations. The use of generalist LLMs results in “Syntactic Fabrication” and “Version Pollution” problems because the system combines commands from different operating systems which results in failed configuration attempts. The research introduces a Version-Aware Retrieval-Augmented Generation (RAG) architecture that enables accurate CLI generation across different systems which includes Cisco IOS-XE and Juniper JunOS. The system employs “Hierarchical Knowledge Grounding” which restricts LLM generation to vendor-provided documents that filter content through metadata and tie it to specific operating system versions. The system includes a “Syntactic Verification Gate” which uses Batfish static analysis to identify errors that would occur during deployment. The evaluation of multi-vendor intent benchmarks shows that version-locked grounding achieves zero hallucinations (0% vs. 4.3% baseline) and an 87.0% Syntactic Validity rate. The study shows that 17.4% of Critical Failure Rate (CFR) errors remain which include hidden semantic mistakes that pass syntax tests but do not achieve operational objectives. The system creates a “Grounding Audit Trail” which establishes a 49,000:1 cognitive compression ratio that allows human reviewers to spend less time searching through documentation. The research indicates that AI-based synthesis systems cannot operate without human control which leads to the implementation of an “Audit-Ready” framework that improves expert human monitoring.

Keywords: *IBN* , *LLMs* , *RAG* , *CLI* , *Hallucination* , *Batfish* , *Verification*

ACKNOWLEDGEMENT

This project work would not have been possible without the contribution of many people. It gives me immense pleasure to express my profound gratitude to our honorable Chancellor **Sri Mata Amritanandamayi Devi**, for her blessings and for being a source of inspiration. I am indebted to extend our gratitude to our Administrative Director, **Swami Vinayamritananda Puri** and Campus Director, **Mr. I B Manikantan** Amrita School of Computing and Engineering, for facilitating us all the facilities and extended support to gain valuable education and learning experience.

I register my special thanks to **Dr. V. Jayakumar**, Principal Amrita School of Computing and Engineering for the support given to me in the successful conduct of this project. I wish to express my sincere gratitude to **Dr. S. Baghavathi Priya**, Chairperson, **Dr. S. Sountharrajan**, Associate Professor, **Dr. S. Natarajan**, Program Chair and to my supervisor **Mrs. V Nivethitha**, Department of Computer Science and Engineering for their inspiring guidance, personal involvement and constant encouragement during the entire course of this work.

I am grateful to Project Coordinator, Review Panel Members and the entire faculty of the Department of Computer Science & Engineering, for their constructive criticisms and valuable suggestions which have been a rich source to improve the quality of this work.

Aryan Madhusudhanan
(CH.SC.U4CSE23207)

Srivishnu T
(CH.SC.U4CSE23249)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	Abstract	iv
	Acknowledgement	v
	List of Tables	x
	List of Figures	xi
	List of Symbols and Abbreviations	xii
1	**INTRODUCTION**	1
	1.1 BACKGROUND: THE EVOLUTION OF NETWORK MANAGEMENT	1
	1.2 MOTIVATION	1
	1.3 ROLE OF RAG IN IBN	2
	1.4 THE IBN PARADIGM AND THE NORTHBOUND CHALLENGE	2
	1.5 OVERVIEW OF THE PROPOSED SYSTEM: ADDRESSING THE RELIABILITY CRISIS	2
	1.6 PHASE-WISE RESEARCH OBJECTIVES	3
	1.7 PHASE 1: KNOWLEDGE GROUNDING AND SYNTACTIC SYNTHESIS (CURRENT WORK)	3
	1.8 PHASE 2: DIGITAL TWIN INTEGRATION AND LOGICAL VERIFICATION (PROPOSED)	3
	1.9 PHASE 3: CLOSED-LOOP AUTOMATION AND SELF-HEALING (PROPOSED)	4
	1.10 SIGNIFICANCE OF THE PROJECT	4
	1.11 SCOPE OF THE STUDY	5

2	**LITERATURE REVIEW**	6
	2.1 OVERVIEW	6
	2.2 TAXONOMY OF HISTORICAL EVOLUTION IN NETWORK AUTOMATION	6
	2.3 REVIEW OF PREVIOUS PAPERS	6
	2.4 LITERATURE SUMMARY TABLE	18
	2.5 SUMMARY OF INDIVIDUAL STUDIES	20
	2.6 SYNTHESIS OF TECHNICAL IMPEDIMENTS IN CURRENT PARADIGMS	30
3	**PROBLEM STATEMENT AND METHODOLOGY**	32
	3.1 PROBLEM STATEMENT	32
	3.2 OVERVIEW OF THE PROPOSED SYSTEM	32
	3.3 DATASET DESCRIPTION	33
	3.4 DATASET COMPOSITION	33
	3.5 DATA PREPROCESSING	34
4	**SYSTEM DESIGN AND EXPERIMENTAL WORK**	35
	4.1 SOFTWARE REQUIREMENTS	35
	4.2 SYSTEM ARCHITECTURE OVERVIEW	35
	4.3 STAGE 1: INTENT PARSING (SEMANTIC ABSTRACTION)	36
	4.4 STAGE 2: VERSION-AWARE RAG (METADATA-ANCHORED RETRIEVAL)	36
	4.5 STAGE 3: SYNTHESIS AND GROUNDING	37
	4.6 STAGE 4: BATFISH VERIFICATION GATE	37
	4.7 INTENT PARSER IMPLEMENTATION	38
	4.8 RETRIEVAL ALGORITHM IMPLEMENTATION	38

4.9 EVALUATION METRICS 39

4.10 TESTING STRATEGY 40

4.11 EXPERIMENTAL WORK AND SIMULATION ENVIRONMENT 40

5 RESULTS AND DISCUSSIONS 41

5.1 PERFORMANCE METRICS 41

5.2 EXPERIMENTAL SETUP 41

5.3 QUANTITATIVE PERFORMANCE ANALYSIS 41

5.4 ANALYSIS OF HALLUCINATION REDUCTION 42

5.5 SYNTACTIC IMPROVEMENT VIA BATFISH GATING 43

5.6 ANALYSIS OF SEMANTIC FAITHFULNESS 43

5.7 SAMPLE SYSTEM OUTPUT: END-TO-END EXAMPLE 43

5.8 CASE STUDY: MULTI-VENDOR OSPF SYNTHESIS 44

5.9 CRITICAL FAILURE RATE AND THE SEMANTIC GAP 46

5.10 QUALITATIVE ANALYSIS: THE GROUNDING AUDIT TRAIL 46

5.11 DISCUSSION OF PERFORMANCE IMPROVEMENTS AND REMAINING GAPS 46

5.12 STATISTICAL SIGNIFICANCE ANALYSIS 47

5.13 ERROR DISTRIBUTION ANALYSIS 48

5.14 ABLATION STUDY: COMPONENT-WISE CONTRIBUTION 48

5.15 KNOWLEDGE BASE COVERAGE ANALYSIS 49

5.16 ERROR CASE ANALYSIS: HALLUCINATION DETECTION EXAMPLES 50

5.17 RETRIEVAL PRECISION ANALYSIS 50

5.18 SYSTEM LATENCY ANALYSIS 51

6 CONCLUSIONS AND SCOPE FOR FURTHER WORK 52

6.1 CONCLUSION 52

6.2 FUTURE WORK 53

7 REFERENCES 54

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	LITERATURE SUMMARY TABLE	18
5.1	COMPARATIVE PERFORMANCE RESULTS	42
5.2	EXTRACTED INTENT PARAMETERS	44
5.3	RETRIEVED DOCUMENTATION CHUNKS	44
5.4	STATISTICAL SIGNIFICANCE OF PERFORMANCE IMPROVEMENTS	47
5.5	SYNTACTIC FAILURE DISTRIBUTION	48
5.6	ABLATION STUDY: COMPONENT-WISE PERFORMANCE CONTRIBUTION	48
5.7	KNOWLEDGE BASE COVERAGE BY PROTOCOL CATEGORY	49
5.8	END-TO-END PROCESSING LATENCY BREAKDOWN	51

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	ARCHITECTURE DIAGRAM OF THE PROPOSED SYSTEM	36
4.2	STRUCTUREDINTENT PYDANTIC MODEL SCHEMA	38
4.3	COMPARISON OF DIFFERENT CHUNKING STRATEGIES SHOWING THAT TIERED HIERARCHICAL CHUNKING MINIMIZES LLM HALLUCINATIONS BY PRESERVING CONTEXT.	40
5.1	COMPARATIVE ANALYSIS ACROSS FOUR KEY RELIABILITY METRICS. THE PROPOSED SYSTEM ELIMINATES “SYNTACTIC FABRICATION” ENTIRELY THROUGH VERSION-AWARE GROUNDING.	42
5.2	GENERATED OSPF CONFIGURATION	44
5.3	MULTI-VENDOR OSPF INTENT	45
5.4	GENERATED CISCO IOS-XE 17.6 CONFIGURATION	45
5.5	GENERATED JUNIPER JUNOS 22.4 CONFIGURATION	45

LIST OF SYMBOLS AND ABBREVIATIONS

ACL	-	Access Control List
API	-	Application Programming Interface
AS	-	Autonomous System
BGP	-	Border Gateway Protocol
CCR	-	Cognitive Compression Ratio
CFR	-	Critical Failure Rate
CLI	-	Command Line Interface
HITL	-	Human-in-the-Loop
HR	-	Hallucination Rate
IBN	-	Intent-Based Networking
IOS-XE	-	Internetwork Operating System (Cisco)
IP	-	Internet Protocol
JSON	-	JavaScript Object Notation
JunOS	-	Juniper Network Operating System
LLM	-	Large Language Model
MD5	-	Message Digest 5
MTU	-	Maximum Transmission Unit
NLP	-	Natural Language Processing
NLU	-	Natural Language Understanding
OS	-	Operating System
OSPF	-	Open Shortest Path First
RAG	-	Retrieval-Augmented Generation
RC	-	Review Coverage
RP	-	Retrieval Precision
SOTA	-	State-of-the-Art
SV	-	Syntactic Validity
VLAN	-	Virtual Local Area Network

CHAPTER-1

INTRODUCTION

1.1 Background: The Evolution of Network Management

Modern communication systems have undergone rapid development which has created networks with unprecedented levels of complicated design. Human operators used to handle all network operations but this method proved to be dangerous because it introduced errors and failed to suit the requirements of large networks that had different types of systems. Intent-Based Networking (IBN) introduces a new networking paradigm that enables automatic conversion of high-level natural language goals into network device settings. The modern IBN systems gained new capabilities because they now use Large Language Models (LLMs) and Small Language Models (SLMs) to analyze unstructured human language. The system now uses Retrieval-Augmented Generation (RAG) to establish trustworthy AI translation through vendor documentation which improves its translation accuracy.

1.2 Motivation

Generalist artificial intelligence models have potential yet they function as black boxes which produce hallucinations and experience “negative transfer” when used in telecommunications. The general linguistic principles fail to explain the strict rules which govern network syntax and logic. The existing RAG-enabled systems fail to handle version-specific requirements which causes “Version Pollution” because they mix commands from different OS releases. The operators cannot implement AI-generated configurations into production systems because the system lacks both precise syntactic validation and complete audit trail verification. Autonomous network synthesis faces a major obstacle because of this reliability problem.

1.3 Role of RAG in IBN

The project introduces a Version-Aware RAG architecture to achieve a complete solution that connects high-level intent with deployable configuration. The system achieves accurate Command Line Interface standard scripts through its strict LLM generation framework which uses version-locked vendor manuals as its base. The framework uses a conversational feedback loop to fix semantic problems before it generates output and it employs deterministic verification tools like Batfish to validate cross-vendor system compatibility and syntactic accuracy. The system creates an open documentation system that provides complete traceability for audits, which decreases hallucinations while building operator trust in critical networking situations.

1.4 The IBN Paradigm and the Northbound Challenge

Intent-Based Networking (IBN) represents the next stage of this evolution. The operators use business goals which they define at a high level to create their operational procedures. The core challenge of IBN exists within its central translation component. The process of converting unstructured natural language into network operating system (NOS) syntax requires complete understanding of word meanings and technical accuracy across different vendor systems.

1.5 Overview of the Proposed System: Addressing the Reliability Crisis

The proposed system is a reliability-first synthesis engine designed to bridge the "Northbound" gap while eliminating the inherent fallibility of generalist LLMs. The architecture identifies and resolves two primary failure modes:

1. **Syntactic Fabrication**: Where the model generates plausible but non-existent keywords or parameters that do not exist in the vendor's grammar.
2. **Version Pollution**: Where the model erroneously retrieves valid syntax from one OS release (e.g., Cisco IOS 15.x) and applies it to another (e.g., IOS-XE 17.6), leading to configuration rejection.

By implementing a **Version-Aware Retrieval-Augmented Generation (RAG)** pipeline, our system locks the LLM's knowledge grounding to specific vendor firmware releases. This is

followed by a deterministic **Syntactic Verification Gate** using Batfish static analysis, ensuring that only grammatically correct and logically sound configurations are presented to the operator.

1.6 Phase-Wise Research Objectives

The project is structured into three distinct developmental phases to ensure a transition from static configuration synthesis to a fully autonomous, self-healing network ecosystem.

1.7 Phase 1: Knowledge Grounding and Syntactic Synthesis (Current Work)

The primary objective of this phase is to eliminate the probabilistic fallibility of LLMs through documentation-anchored retrieval:

- **Objective 1: Develop a Version-Aware Knowledge Base**: Constructing a high-fidelity vector store of 10,000+ chunks from official Cisco and Juniper documentation, tagged with strict OS version metadata.
- **Objective 2: Implement Hierarchical Knowledge Grounding**: Ensuring the LLM (Llama 3.1 8B) is restricted to generating CLI commands exclusively from retrieved, version-locked context.
- **Objective 3: Establish a Deterministic Verification Pipeline**: Integrating Batfish static analysis to catch syntax errors and "off-by-one" parameter mistakes before deployment.
- **Objective 4: Provide a Grounding Audit Trail**: Measuring performance across Syntactic Validity (SV), Hallucination Rate (HR), and Cognitive Compression Ratio (CCR) with command-level citations.

1.8 Phase 2: Digital Twin Integration and Logical Verification (Proposed)

The second phase shifts focus from syntax to semantics and operational state:

- **Objective 1: Digital Twin Realization**: Implementing a virtualized network environment using **GNS3** or **Eve-NG** to mirror the target production topology.
- **Objective 2: Runtime Logical Validation**: Executing synthesized CLI in virtualized topologies to verify operational states and protocol stability.

- **Objective 3: State-Based Correction**: Detecting "silent" semantic errors, such as routing loops, that static analysis alone cannot identify.
- **Objective 4: Behavioral Compliance**: Ensuring behavioral compliance by comparing GNS3 simulation outcomes against the original intent.

1.9 Phase 3: Closed-Loop Automation and Self-Healing (Proposed)

The final phase aims for autonomous lifecycle management:

- **Objective 1: Structured Log Parsing**: Developing a Structured Log Parser to translate raw GNS3/Batfish error logs into actionable feedback for the LLM.
- **Objective 2: Contextual Retry Mechanism**: Implementing a contextual retry mechanism that feeds specific failure reasons back to the engine for targeted "patching".
- **Objective 3: Prompt Refinement Optimization**: Optimizing prompt refinement to guide the LLM in fixing specific configuration blocks without full re-synthesis.
- **Objective 4: Critical Failure Rate Reduction**: Reducing the 17.4% Critical Failure Rate (CFR) toward 0% through iterative, feedback-led refinement.

1.10 Significance of the Project

This project holds significant value for both academic research and industrial network operations:

- **Operational Safety**: By providing a "Grounding Audit Trail," the system allows human operators to verify synthesized commands against official manual pages, reducing the cognitive load of documentation search by a factor of 49,000.
- **Vendor Neutrality**: The modular architecture is designed to support heterogeneous environments (Cisco, Juniper, etc.), facilitating easier management of multivendor data centers.
- **Cost Reduction**: Automating the "Northbound" translation reduces the time required for configuration deployment from hours to seconds while minimizing the risk of expensive manual-error-driven outages.

1.11 Scope of the Study

This research focuses on the “Synthesis and Verification” phase of the IBN lifecycle. The study is scoped to:

- **Target OS Platforms**: Cisco IOS-XE (Release 17.6) and Juniper JunOS (Release 22.4).
- **Network Protocols**: OSPF (Open Shortest Path First) and BGP (Border Gateway Protocol) routing configurations, VLAN management, and standard/extended ACLs.
- **Toolchain**: Utilizes Llama 3.1 (8B) for generation, ChromaDB for vector storage, and Batfish for static analysis.
- **Validation Constraints**: Synthesis is validated against a Batfish parsable subset; complex BGP attributes rely on secondary regex or manual review.

CHAPTER-2

LITERATURE REVIEW

2.1 Overview

The academic field of network orchestration and Intent-Based Networking (IBN) has experienced major changes during the last ten years which brought new developments beyond its original rule-based systems and now uses advanced Artificial Intelligence systems. The research investigates three main technical areas which include the development of IBN orchestration systems and the application of Retrieval-Augmented Generation (RAG) for technical information handling and the new abilities of Large Language Models (LLMs) to perform accurate intent translation.

2.2 Taxonomy of Historical Evolution in Network Automation

The development of network automation research started with imperative scripting and advanced through the use of declarative orchestration. Initial research work dedicated its efforts to creating Domain-Specific Languages (DSLs) and templating engines yet these systems maintained their fixed syntactical structure which required extensive human effort for their smooth functioning as network infrastructure expanded.

2.3 Review of Previous Papers

2.3.1 The Evolution of Intent-Based Networking (IBN) and Policy Translation

The practice of Intent-Based Networking (IBN) has transformed network management by replacing manual configuration methods with automated systems that follow predefined policies. The initial research attempts aimed to eliminate the severe operational difficulties which emerged from security solutions that used trial-and-error methods. Bringhenti et al. [1]

conducted a survey to examine current methods which automate network security settings because manual network control in distributed systems leads to operational disruptions. The research team assessed 95 primary studies through their method which used Software-Defined Networking (SDN) and Network Function Virtualization (NFV) and mathematical optimization techniques including Integer Linear Programming (ILP) and MaxSMT. The research demonstrated that advanced optimization techniques together with formal verification methods enabled system expansion to handle networks consisting of 100000 devices and 4000 active security regulations. The examined frameworks have a research gap that needs to be addressed because they fail to handle operating system versioning which results in "Version Pollution" when commands from different firmware versions create conflicts. The systems do not include Batfish which provides deterministic syntactic verification for secure testing of old CLI translations and they completely lack command-level human monitoring records which show deployment processes.

Boutouchent et al. [2] introduced AMANOS as a solution for operating cloud-native 5G systems together with edge computing systems through their introduction of an automated intent-based management system. The authors aimed to solve the prohibitive operational expenditures associated with manually integrating computing and communication resources at the network edge. The system architecture requires a Natural Language Understanding (NLU) module which integrates Named Entity Recognition (NER) technology together with a machine learning marketplace to convert business intents into active Kubernetes deployments. AMANOS achieved service continuity through its execution on a physical testbed for 360-degree video streaming which maintained a 10-12 Mbps throughput while using only 0.6% of its computational capacity. The framework successfully functions according to its operational requirements but research within this framework encounters major limitations because it does not consider OS versioning updates which exist between different 5G core components. The system lacks deterministic syntactic verification which would enable validation of its generated

Helm charts during structural integrity assessment while it requires command-level grounding together with audit trails to trace probabilistic NLU hallucinations back to their origin.

To close the gap between high-level user requests and low-level policy conflicts, Njah et al. [3] developed an AI-Driven Intent-Based Network Architecture (IBNA). They addressed the specific problem of the complex math involved in resolving inter-policy conflicts in dynamic, dense IoT ecosystems like Healthcare 4.0. Their approach uses a fine-tuned BERT model for precise intent refinement and employs Multi-Agent Deep Reinforcement Learning (MADRL) to automatically resolve spatial configuration conflicts during intent activation. Testing on datasets from 5,000 to 40,000 intents, the fine-tuned BERT model achieved nearly perfect entity extraction and resolved conflicts in about 15,000 seconds for blocks of 5,000 intents. However, notable research gaps limit its real-world use. The framework completely ignores OS versioning for its target SDN controllers, risking serious Version Pollution during firmware updates. It also lacks deterministic syntactic verification, like Batfish, and relies on probabilistic reinforcement agents without strict command line syntax checks. Additionally, it fails to provide verifiable command-level human audit trails.

As network models move toward 6G, researchers need to create systems that balance high technical performance with social values. Kerboeuf et al. [4] addressed this issue by proposing a detailed design method for 6G End-to-End (E2E) systems from the Hexa-X-II view. The researchers used architectural plans, digital twins, and virtual E2E simulation modeling to connect top-down intent management with bottom-up hardware feasibility. They validated Key Performance Indicators (KPIs) by evaluating a simulated collaborative robot (cobot) use case. These KPIs included sub-10ms E2E latencies and emerging Key Value Indicators (KVIs) like sustainability and trustworthiness. However, a significant research gap is that the framework does not handle OS versioning management across distributed, multi-vendor edge nodes, which makes it vulnerable to Version Pollution. Additionally, the plan lacks reliable syntactic verification methods to ensure that AI-generated dynamic changes are structurally

sound before execution, and it does not provide the necessary command-level audit trails for zero-touch observability.

The use of natural language to deploy intents provides operators with operational flexibility, but network staff members maintain distrust toward AI-based deployment systems. Jacobs et al. [11] developed a machine learning-based Ambiguity Detection module which extends the LUMI conversational networking framework to solve this problem. The methodology uses a Random Forest Classifier to examine intents that have been converted into the **Nile** language, which enables the system to identify both semantic and operational conflicts between new requests and existing deployment policies. The system demonstrated its ability to scale when it achieved an F1 score of 0.947 while detecting operational ambiguities in synthetic datasets containing 5,000 intent pairs and the actual university policy dataset from the **campi** database. The research established a fundamental trust framework for operators, but significant research gaps still exist. The system shows explicit vulnerabilities to terms which are not part of its vocabulary yet it does not support the operating system versioning system, which leads to Version Pollution in its generic intents whenever they get mapped to different vendor firmware. The architecture design does not include Batfish-style deterministic syntactic verification to validate its final legacy CLI output while it cannot generate permanent, command-level audit trails which would able to show its internal processing logic.

The practice of disclosing confidential network infrastructure information to main control systems in multiple-administrator environments creates a breach of essential data protection regulations. The authors Hussein and Ibnkahla [19] solved this problem by developing the Harmony Slice Master H-SliceMaster which serves as an advanced intent-based network slicing system designed specifically for IoT networks. The proposed methodology includes the Promise and Price Network Operation PPNO system which employs Double Dueling Deep Q-Networks D3QN and Soft Network Control SNC to achieve optimal resource allocation from one domain to another while maintaining domain configuration secrecy. The D3QN PPNO approach maintained 2ms delay limits for smart city traffic simulation on the Nobel-Germany

SNDlib backbone topology while achieving 50% cost savings compared to existing greedy algorithms. The privacy-protecting system maintains essential research weaknesses at its core. The system shows complete unawareness of all firmware restrictions which apply to different IoT sensors while it neglects to handle operating system versioning thus creating a risk of Version Pollution. The system requires deterministic syntactic verification for translated SDN policies before execution but it does not provide command-level audit trails which can trace the DRL agent's pricing strategies.

Recognizing the gap between AI-driven resource allocation and secure multi-domain data management, Hussein and Ibnkahla [5] conducted a thorough survey to define the Intelligent Intent-Based Network Slicing (I-IBNS) ecosystem. The main issue they examined was the industry's struggle to combine natural language intent translation with proactive, privacy-focused IoT network slicing. By reviewing how Deep Reinforcement Learning (DRL) and federated learning fit into Network Data Analytics Functions (NWDAF), the study highlighted performance metrics that showcase sub-millisecond end-to-end delay synchronizations and Service Level Agreement (SLA) guarantees. While mapping the connection of these technologies, the survey points out significant research gaps in the current literature. The I-IBNS frameworks they reviewed fail to address OS versioning across millions of diverse IoT sensors, which leads to Version Pollution. Additionally, they lack the ability to perform deterministic syntactic verification (like Batfish) to accurately manage high-level intent translations, relying instead on probabilistic AI models that do not provide clear, command-level audit trails for network operators.

2.3.2 Large Language Models and Agentic AI in Network Orchestration

The integration of Large Language Models (LLMs) into IBN has changed policy translation, but rigid template-based methods still struggle to meet new user demands. Kou et al. [12] addressed this issue by developing the Generative Intent Abstraction (GIA) framework, which completely avoids static templates. The main approach uses GPT-4 with enhanced prompting to

automatically generate Python code with the NetworkX library, creating flexible intent graphs. Intent negotiation is then framed as a Constrained Subgraph Isomorphism Problem (SIP) using the VF2 algorithm. Tested on a custom Network Intent Dataset sorted by complexity, the GIA framework reduced code hallucinations and achieved an impressive 180% increase in F1-score when handling new intents compared to traditional methods. However, the framework has significant gaps. It considers the physical layer as a generic topology graph and overlooks OS versioning, leading to Version Pollution. Additionally, it does not provide deterministic syntactic checks (like Batfish) to ensure that the generated Python graphs compile into valid legacy CLI, and it lacks clear command-level audit trails.

Managing the entire lifecycle of network intent across different 5G domains is very complex due to the unclear nature of natural language. Mekrache et al. [13] tackled this challenge by suggesting an end-to-end, LLM-focused system to handle multi-domain Cloud/Edge and Radio Access Network (RAN) configurations. The system uses the 34-billion parameter Code Llama model, integrating few-shot learning, human feedback, and MPNet v2 sentence transformers to break down natural language intents and convert them into standardized Infrastructure-Level Intents (ILIs) like Network Service Descriptors (NSDs). Deployed on the EURECOM 5G testbed, the system produced structurally correct JSON files but faced significant latency issues, with end-to-end translation times exceeding two minutes when users requested more than three applications at once. A critical gap in this LLM-centered approach is its failure to address OS versioning for specific RAN hardware, which risks serious Version Pollution. Moreover, the system does not have autonomous deterministic checks (like Batfish), mistakenly assuming that the raw JSON output from the LLM is ready for deployment, and it does not provide reliable, command-level audit trails.

In 6G environments with multiple stakeholders, coordinating the smooth transition of mobile collaborative robot (cobot) services without disrupting important operations is a major challenge. Orozco et al. [14] tackled this issue by creating a closed-loop automation proof-of-concept that follows ETSI Zero-Touch Network and Service Management standards. The

method uses an Intent-Based Network Intent Management Entity (IBN-IME) supported by Natural Language Processing (NLP) interfaces and Hierarchical Finite State Machines (FSM) to control cobot operations through cloud-virtualized ROS nodes. Validated in a physical lab testbed, the setup effectively orchestrated intent instantiations, aiming to minimize service downtime to nearly zero during autonomous 360-degree surveillance video transfers. Nonetheless, this architecture reveals important research gaps. It does not address OS versioning, making it highly prone to Version Pollution when moving containers between edge nodes with different kernels. The system also misses deterministic checks to validate the NLP-translated intents structurally and does not create command-level audit trails to counteract potential AI hallucinations.

Routing every intent through large proprietary LLMs incurs high operational costs, delays, and energy use in data-plane network programming. Angi et al. [15] tackled this sustainability issue by introducing LLNet, which uses very efficient Small Language Models (SLMs) to manage intent translation locally. The architecture leverages the 3.8-billion parameter Phi-3-mini SLM, enhanced with few-shot prompting, to translate natural language intents into intermediate JSON structures. A deterministic Network Manager then compiles these into P4, OpenFlow, and eBPF codes. Tested on a synthetic dataset of 30,000 JSON networking intents using the Damerau-Levenshtein edit distance metric, the SLM achieved over 88% translation accuracy while cutting operational API and token costs by 89.54% compared to larger models. Despite these benefits, LLNet has serious research gaps. It assumes static application programming interfaces and does not address OS versioning or firmware drift (Version Pollution). Additionally, the pipeline ends at JSON generation without including deterministic verification (like Batfish) to validate the final compiled legacy CLI, and it lacks clear, command-level audit trails.

Foundation models often struggle with the strict syntax indentation needed for generating valid Infrastructure-as-Code (IaC) files, creating a significant hurdle to autonomous orchestration. Brodimas et al. [18] addressed this challenge by developing a distributed multi-agent

system based on the Model Context Protocol (MCP) and LangChain. To lower operational costs, the researchers fine-tuned open-source Small Language Models (SLMs)—specifically Mistral-7B and Qwen3-0.6B—using Quantized Low-Rank Adaptation (QLoRA) to generate accurate Kubernetes kubectl commands. Evaluated against a dataset of 35,000 configuration examples, the fine-tuned Mistral model achieved remarkable code generation similarity scores near 90% for BLEU and 80% for METEOR, successfully matching proprietary model performance. However, local SLMs faced abrupt bottlenecks at their 8K token context limit. Crucially, the framework has obvious research gaps: it ignores OS versioning for evolving Kubernetes APIs, risking Version Pollution. Additionally, it lacks strict deterministic checks to formally validate the generated YAML files before deployment and depends on opaque routing without offering solid, command-level human audit trails.

The absence of a universal conversational intelligence system which can integrate all aspects of network design and troubleshooting and configuration tasks compels operators to use separate AI systems which require extensive manual work. Huang et al. [20] solved this problem by developing ChatNet which serves as an architectural framework that enables LLMs to function as “Network Embodied Intelligence” systems that can access external solver APIs. The system uses GPT-4 as its core component which combines three technologies to create four distinct operational functions that include an Analyzer a Planner a Calculator and an Executor. The study demonstrated that ChatNet could perform complex capacity planning tasks through its autonomous capabilities because the model achieved scores above 0.8 with little need for human support which experts evaluated using standardized scoring methods. The study identified critical research deficiencies because the LLM consistently produced false mathematical results that looked like exact mathematical solutions. The system does not provide any solutions to OS versioning problems which makes it vulnerable to Version Pollution. The system does not include deterministic syntactic verification which Batfish provides to validate CLI/ACL code output for secure deployment and it cannot produce command-level audit trails for verification purposes.

2.3.3 Retrieval-Augmented Generation (RAG) and Domain-Specific Knowledge Management

Language models based on generic language models are very prone to serious hallucinations in the event of parsing thick, proprietary industrial documentation, and therefore it cannot be easy to have technicians accessing specific repair protocols on short notice. This was solved by Chen et al. [6], who designed Interactive Industrial Knowledge Management (IIKM) system which clearly adapts RAG to the domain-specific industrial settings. The approach combines GPT-3.5 and GPT-4 with a highly personalized pipeline, which means that the TF-IDF algorithm is applied to document chunks with the help of the BM25 algorithm and a BAAI cross-encoding reranker. The optimized RAG system using private 234 MB data of Sakura Inc. product manuals and organizational regulations for evaluation gave a Mean Reciprocal Rank (MRR) of 97.97% and a Recall of 91.62%, which was vastly superior to the conventional embedding methods. Although BAAI reranker was able to decrease hallucinations in text generation, there is still considerable research vacuum in hardware-focused implementation. The system is also totally unaware of OS versioning of the equipment manuals, which poses hazardous Version Pollution where different firmware manuals coincide. In addition, it does not have deterministic syntactic validation to mathematically verify any of the generated technical commands, and does not have command-level human audit trails that are immutable.

Massive heterogeneity of hardware impedes the automation of wireless networks heavily through LLMs, and the standard RAG pipelines fetch too much noise in long telecom manuals, utterly to the point of overload the generation process. Wang et al. [7] have overcome this difficulty by suggesting LIT, an intent translation framework that introduces a new algorithm of Hierarchical Index Retrieval (HIR). The approach varies the methodology by utilizing HIR with a prompt compression algorithm to drive clean context to a fine-tuned ChatGLM-6B model and a Deep Reinforcement Learning (DRL) Mixture of Experts (MoE) methodology which dynamically generates parameter at the metric level by considering network states in

real-time. Intent transformation F1 scores using custom datasets of 1,600 intent mappings and larger topologies such as Intellifiber were benchmarked 57.5% higher and total network QoS revenue increased by up to 56.95. In spite of such optimizations, the framework has deep gaps in research. The hierarchical indexing is a manual indexing that is not dynamic enough to deal with versioning of OS and live firmware updates and is assuring Version Pollution. Also, the architecture does not support deterministic syntactic verification (in the style of Batfish) to strictly gate the generated policies, or give transparent command-level audit trails to describe the derivation of the parameters of the MoE.

In the Internet of Medical Things (IoMT), Multimodal Large Language Models (MLLM) cannot render proper diagnostics because of the absence of new and multimodal data because of rigorous privacy policies at hospitals. Su et al. [8], have discussed this vulnerability of freshness of data by recommending a hybrid RAG-empowered MLLM framework, which has been secured using cross-chain blockchain technology. The authors used a Generative Diffusion Model (GDM) that is implemented on top of Deep Reinforcement Learning to optimize data-sharing contracts among hospitals and the Age of Information (AoI) metric to measure the freshness of data. The GDM-based contract scheme was evaluated with the LLaVA-Med-10K/60K dataset and achieved 20.35% higher overall utility than standard RL baselines, and the hybrid multimodal RAG also showed a significant improvement in the quality of the diagnostic provided by the MLLM by filtering unimodal search results. Nevertheless, in terms of networking, there are still important gaps in research. This framework has failed entirely to resolve the issue of OS versioning of the IoMT sensors that produce the multimodal data, which introduce Version Pollution vulnerabilities. It does not also have deterministic syntactic verification to check any network level commands evoked by the MLLM, or offer any verifiable command level audit trails mapping the inner reasoning of the MLLM.

The development of strict mathematical optimization models of the future wireless networks is a highly manual endeavor, and even the standard LLMs go on a spree when faced with the creation of strict mathematical constraints. Zhang et al. [9] that addressed this

bottleneck, by presenting Interactive AI (IAI) framework which combines GPT-4 with RAG and Chain-of-Thought (CoT) reasoning. The architecture runs on LangChain, and it uses Proximal Policy Optimization (PPO) algorithms to engage network designers with the goal of transforming natural language into complex mathematical representations (e.g., in RIS-assisted SWIPT networks). The system showed exceptional performance at the use of RAG chunk sizes, 1,000 of tokens, to create the correct optimization constraints with fewer than one human intervention needed per task. However, the research identified serious gaps in the study. The LLM experienced disastrous context loss and hallucination at RAG chunk size greater than 4,000 tokens. This architecture completely disregards OS versioning of the underlying mathematical solver tools (Version Pollution), uses human-in-the-loop interventions as opposed to automated deterministic syntactic verification (such as Batfish), and does not have structured human audit trails in a command-level form.

The mathematical complexity and lack of accuracy of bridging the semantic gap between abstract high-level intents and low-level Key Performance Indicators (KPIs) necessary to deploy 5G cellular is mathematically complex and subject to error. The author Mehmood et al. [16] addressed this intent completion issue by dynamically combining Intent-Based Networking with Knowledge Graph Embeddings (KGE). The model of the methodology represents intents with the usage of a Gaussian embedding model (KG2E) and Named Entity Recognition with Hidden Markov Models (HMM). In order to guarantee compliance in the deployment, the system creates closed loop optimization with SPSA and MGDA algorithms. In a simulated 5G Non-Standalone setup, the KG2E model was tested to be able to predict missing service parameters successfully at 91.18 Hits at 10, and the closed-loop optimization was able to cause strict-latency intents to converge within 1600ms. Although KGs managed to organize the data, there are deep gaps in the research. The system presupposes fixed RDF triples, which do not serve OS versioning there between dynamically updated RAN/Core firmware, and causes Version Pollution. Moreover, it does not have deterministic syntactic verification to check

the inferred parameters before it runs and uses opaque high-dimensional Gaussian embedding spaces without giving audit trails at the command level.

Deploying generative intelligence at the telecommunications edge is severely constrained by hardware limits, while highly efficient Small Language Models (SLMs) suffer from short context windows and technical jargon hallucinations. Alabbasi et al. [10], bridged this gap with TeleOracle, a telecom-specialized RAG system powered by a LoRA fine-tuned Phi-2 SLM. To overcome context limitations, the architecture leverages the SelfExtend algorithm to mathematically expand the context window from 2048 to 8192 tokens. Furthermore, it employs a two-stage retrieval process utilizing Semantic Chunking, a Hybrid Search Retriever (semantic plus BM25 keyword), and a Cross-Encoder Reranker. Benchmarked against 10,000 TeleQnA questions and 550 chunked 3GPP documents, the optimized SLM boosted baseline accuracy by 30% to reach 81.20%, remarkably outperforming GPT-4o in strict factual Faithfulness (78.80%). Despite its success in technical QnA, critical research gaps prohibit its use in active network management. TeleOracle fails to tie its retrieval to specific OS versioning implementations (Version Pollution), lacks deterministic syntactic verification (like Batfish) to ensure its outputs securely compile into executable CLI, and provides no operational command-level audit trails.

Prior to recent benchmarking efforts, the telecommunications field lacked a standardized mechanism to rigorously assess an LLM's true factual proficiency regarding dense, standard-specific networking protocols. Maatouk et al. [17], addressed this absence by creating the TeleQnA benchmark dataset. The methodology employed an automated, adversarial GPT-3.5 generator/validator loop, supplemented by K-Means clustering and multi-stage human expert validation, drawing from 25,000 pages of scholarly and 3GPP standards publications. Evaluating top models on the resulting 10,000-question dataset revealed that while GPT-4 achieved a 74.91% overall accuracy—rivaling active human professionals in general knowledge—it failed abruptly on complex mathematical standards specifications without the aid of RAG context. This study exposes a glaring research gap in generic model training. However, the

dataset methodology itself fails to account for specific OS versioning or vendor implementations, evaluating generalized standards and risking Version Pollution in practical application. Furthermore, the framework completely lacks deterministic syntactic verification components for evaluating generated network code, and the black-box nature of the LLM validator provides no verifiable command-level human audit trails.

2.4 Literature Summary Table

No	Authors and Year	Paper Title	Journal Name	Methods Used	Key Findings	Research Gaps
1	Bringhenti et al. (2024)	Automation for Network Security Configuration	ACM Computing Surveys	Formal verification, MaxSMT, ILP optimization.	Categorizes formal policy translation to NSF rules.	Lacks a Syntactic Validator for vendor-specific CLI tokens.
2	Boutouchent et al. (2024)	AMANOS: An Intent-Driven Management and Orchestration System	IEEE Communications Magazine	NLU, Ontology-based models, Machine Learning.	Proposes closed-loop orchestration for 5G Core.	Fails to address physical hardware Syntax Drift.
3	Njah et al. (2024)	An AI-Driven Intent-Based Network Architecture	IEEE Communications Magazine	LLM fine-tuning (BERT), NER, MADRL.	BERT-based intent refinement and activation logic.	Targets SDN policies rather than Native CLI synthesis.
4	Kerboeuf et al. (2024)	Design Methodology for 6G End-to-End System	IEEE Open Journal of the Communications Society	Architectural blueprinting, E2E system modeling.	Outlines blueprint for 6G intent-driven automation.	Ignores Firmware-level Syntax Drift at the network edge.
5	Haj Hussein et al. (2025)	Intelligent Intent-Based Network Slicing for IoT Systems	IEEE Transactions on Network and Service Management	Literature survey, taxonomy definition.	Vision for IoT slice management and orchestration.	Overlooks the Syntactic Synthesis for multi-vendor devices.
6	Chen et al. (2025)	Application of RAG for Industrial Knowledge Management	Computer Standards & Interfaces	RAG, BM25, Cross-encoder reranker, GPT-4.	Metadata-aware RAG for industrial technical manuals.	Generates NL text, not machine-executable code.
7	Wang et al. (2025)	Hierarchical Index Retrieval RAG-Driven Intent Translation	IEEE Transactions on Mobile Computing	RAG with HIR, Mixture of Experts (MoE).	Combined RAG with hardware manuals for policy.	Lacks a Deterministic Syntactic Validator for tokens.

No	Authors and Year	Paper Title	Journal Name	Methods Used	Key Findings	Research Gaps
8	Su et al. (2024)	Hybrid RAG-Empowered Multimodal LLM for Secure Data Management	IEEE Internet of Things Journal	Hybrid Multimodal RAG, Cross-chain technology.	Secure medical IoT data management via RAG.	Addresses data privacy, not Intent-to-CLI translation.
9	Zhang et al. (2024)	Interactive AI With RAG for Next Generation Networking	IEEE Network	Interactive AI, RAG, Chain-of-Thought (CoT).	RAG-assisted AI for network optimization problems.	Assistant for math; generates no structural CLI configs.
10	Alabbasi et al. (2025)	TeleOracle: Fine-Tuned RAG With Long-Context Support	IEEE Internet of Things Journal	RAG with SLM (Phi-2), LoRA, SelfExtend.	SLM-based RAG aligned to 3GPP for technical Q&A.	Strictly for Q&A; lacks a Synthesis Engine.
11	Jacobs et al. (2025)	Establishing Trust for Using Natural Language for Intent-Based Networking	IEEE Transactions on Network and Service Management	NER (Bi-LSTM), Ambiguity Detection, Nile Policy.	LUMI pipeline for NL-to-Policy via Nile.	Compiles to intermediate languages rather than Native CLI.
12	Kou et al. (2025)	GIA: LLM-Enabled Generative Intent Abstraction	IEEE Transactions on Cognitive Communications and Networking	LLM code generation, Subgraph isomorphism.	LLM-enabled structural policy graphs for networks.	Lacks Version-Aware Grounding for specific tokens.
13	Mekrache et al. (2024)	Intent-Based Management of Next-Generation Networks	IEEE Network	LLM (CodeLlama), Few-Shot Learning, RAG.	NL-to-JSON translation for 5G descriptors.	Targets JSON objects; provides no Last-Mile syntax.
14	Orozco et al. (2025)	Intent-Based Service Provisioning for Cobots	IEEE Transactions on Network and Service Management	Intent decomposition, Hierarchical state machines.	Business intent to-3GPP JSON object translation.	Stops at JSON level; fails to account for Syntax Drift.
15	Angi et al. (2025)	LLNet: Instructing Softwarized Devices Using SLM	IEEE Transactions on Network and Service Management	Small Language Models, QLoRA, Few-shot.	SLM-based approach for instructing programmable devices.	Targets abstract APIs; ignores CLI Syntactic Volatility.

No	Authors and Year	Paper Title	Journal Name	Methods Used	Key Findings	Research Gaps
16	Mehmood et al. (2025)	Knowledge-Driven Intent Life-Cycle Management	IEEE Transactions on Network and Service Management	Knowledge Graph Embedding (KG2E), NER.	KGs for metadata-aware intent life-cycle.	Lacks Token-level granularity for CLI synthesis.
17	Maatouk et al. (2025)	TeleQnA: A Benchmark Dataset to Assess LLMs Telecommunications Knowledge	IEEE Network	Automated QnA generation, Text embeddings, K-Means.	Established a 10,000-question benchmark for LLM telecom proficiency.	Focuses on evaluation; lacks a Synthesis Engine for 'Last-Mile' CLI execution.
18	Brodimas et al. (2025)	Intent-Based Infrastructure Orchestration Using Agentic-AI	IEEE Open Journal of the Communications Society	Multi-Agent System, Model Context Protocol.	Multi-agent framework for Infrastructure-as-Code.	Generates K8s syntax vs. Legacy Physical CLI.
19	Haj Hussein et al. (2025)	Privacy-Preserving Intelligent Intent-Based Slicing	IEEE Internet of Things Journal	Deep Q-Networks (DQNs), PPNO.	Privacy-preserving slicing for cross-domain IoT.	Lacks Low-level CLI Accuracy for edge gateways.
20	Huang et al. (2024)	LLMs for Networking: Applications & Challenges	IEEE Network	Domain-adapted fine-tuning, RAG, CoT.	Surveyed LLM techniques for capacity planning.	Lacks a Syntactic Validator for safe hardware execution.

Table 2.1: Literature Summary Table

2.5 Summary of Individual Studies

1. Automation for Network Security Configuration

Bringhenti et al. aim to systematically review the state-of-the-art in automating network security configurations to transition the industry away from manual, error-prone deployments. Their methodology analyzes 95 primary studies focusing on Software-Defined Networking (SDN), Network Function Virtualization (NFV), and mathematical optimization algorithms like Integer Linear Programming (ILP). The study found that advanced optimization and formal verification significantly enhance system scalability, enabling the automated management of topologies with up to 100,000 devices. However, a critical research gap is their failure to address OS versioning across multi-vendor hardware, leading to potential "Version Pollution" where generic

configurations clash with underlying firmware updates. Furthermore, the surveyed frameworks lack deterministic syntactic verification (such as Batfish) to formally gate generated legacy CLI configurations, and they completely omit command-level human audit trails required for deployment transparency.

2. AMANOS: An Intent-Driven Management and Orchestration System

Boutouchent et al. introduce AMANOS, a closed-loop intent-driven management system designed to lower operational expenditures and autonomously manage cloud-native 5G networks and edge computing resources. The architecture employs a Natural Language Understanding (NLU) module based on Named Entity Recognition (NER) and a machine learning marketplace to translate business intents into active Kubernetes deployments. Implemented on a physical testbed for 360-degree video streaming, the system successfully maintained a target 10-12 Mbps network throughput while consuming minimal computational overhead. Despite these achievements, the framework fails to consider OS versioning updates within the 5G core infrastructure, exposing the system to Version Pollution risks during firmware drift. Additionally, the architecture lacks deterministic syntactic verification to rigorously validate the structural integrity of its generated Helm charts and completely omits command-level audit trails to trace probabilistic NLU interpretations back to original operator intents.

3. An AI-Driven Intent-Based Network Architecture

The authors propose an AI-Driven Intent-Based Network Architecture (IBNA) to automate network management and dynamically resolve inter-policy conflicts in ultra-dense environments like Healthcare 4.0. The methodology leverages a fine-tuned BERT model for natural language intent refinement alongside a Multi-Agent Deep Reinforcement Learning (MADRL) framework to ensure intent assurance and conflict resolution during network activation. Evaluated on a custom dataset of up to 40,000 user intents, the fine-tuned BERT model achieved near-perfect entity extraction and rapidly executed spatial conflict resolutions. Nevertheless, the framework entirely ignores OS versioning across the targeted software-defined controllers, leaving the deployment phase highly vulnerable to Version Pollution. It also fundamentally

lacks a deterministic syntactic verification mechanism like Batfish to rigidly gate probabilistic RL agent outputs before deployment, and it provides no verifiable command-level audit trails for human operators.

4. Design Methodology for 6G End-to-End System

Kerboeuf et al. outline a comprehensive design methodology for 6G End-to-End (E2E) systems that balances extreme technical performance requirements with societal values such as sustainability and trustworthiness. Their methodology utilizes architectural blueprinting, digital twinning, and virtual E2E simulation modeling to align high-level intent management with bottom-up hardware capabilities. By evaluating a simulated collaborative robot (cobot) use case, the framework successfully validated strict Key Performance Indicators (KPIs) like sub-10ms E2E latencies alongside emerging Key Value Indicators. A major limitation of this blueprint, however, is its failure to address dynamic OS versioning across distributed, multi-vendor edge nodes, exposing future architectures to Version Pollution. Furthermore, the design lacks concrete deterministic syntactic verification mechanisms to mathematically validate AI-generated reconfigurations prior to execution, and it omits the immutable command-level audit trails necessary for safe zero-touch observability.

5. Intelligent Intent-Based Network Slicing for IoT Systems

Haj Hussein and Ibnkahla conduct a comprehensive survey defining Intelligent Intent-Based Network Slicing (I-IBNS) systems to address the complex management and orchestration challenges of diverse Internet of Things (IoT) applications. The study systematically reviews recent literature on intent-driven resource allocation using artificial intelligence and examines methodologies for privacy-conscious data management across multi-domain IoT ecosystems. The findings highlight that while deep reinforcement learning can optimize radio block allocations and slice scaling, achieving seamless end-to-end data orchestration without violating domain privacy remains a significant hurdle. Crucially, the reviewed I-IBNS frameworks systematically fail to address OS versioning across millions of heterogeneous IoT sensors, making them highly susceptible to Version Pollution. Moreover, the literature strictly lacks determin-

istic syntactic verification like Batfish to formally validate high-level intent translations into physical slice deployments, and it completely omits transparent, command-level audit trails for network operators.

6. Application of RAG for Industrial Knowledge Management

The authors designed an Interactive Industrial Knowledge Management (IIKM) system leveraging Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) to accelerate technical queries and reduce human workload. Their methodology combines GPT-3.5 and GPT-4 with a customized retrieval pipeline featuring TF-IDF, BM25 algorithms, and a BAAI cross-encoding reranker to process complex industrial manuals and regulations. Evaluated on a 234 MB private industrial dataset, the optimized RAG system achieved a Mean Reciprocal Rank (MRR) of 97.97% and effectively mitigated the hallucination issues common in generic LLMs. Despite this high retrieval accuracy, the system is completely blind to OS versioning of the equipment manuals, creating dangerous Version Pollution if instructions for distinct firmware generations overlap. Furthermore, it lacks deterministic syntactic verification to mathematically validate any generated technical commands, and it provides no immutable command-level human audit trails to track the LLM's diagnostic reasoning.

7. Hierarchical Index Retrieval RAG-Driven Intent Translation

Wang et al. propose LIT, an intent translation framework designed to overcome hardware heterogeneity and high dynamic network challenges in wireless networks using LLMs. The system builds a hierarchical index of hardware manuals for RAG and utilizes a Deep Reinforcement Learning-based Mixture of Experts (MoE) approach to dynamically adjust metric-level parameter values according to real-time network status. Tested on simulated topologies like Intellifiber with 1,600 intent mappings, LIT improved intent translation F1 scores by 57.5% and boosted total network revenue by up to 56.95%. Despite these optimizations, the framework's manual indexing fails to dynamically address OS versioning and live firmware updates, ensuring a high risk of Version Pollution. Additionally, the architecture relies on probabilistic MoE

outputs without utilizing deterministic syntactic verification (such as Batfish) to rigidly gate the generated policies, and it fails to provide transparent command-level audit trails.

8. Hybrid RAG-Empowered Multimodal LLM for Secure Data Management

Su et al. tackle data freshness and privacy vulnerabilities in the Internet of Medical Things (IoMT) by developing a hybrid RAG-empowered Multimodal LLM (MLLM) framework secured through cross-chain technology. Their methodology employs a Generative Diffusion Model (GDM) based on Deep Reinforcement Learning to optimize data-sharing contracts, filtering multiple unimodal RAG results using multimodal metrics to enhance the MLLM's inputs. Evaluated using the LLaVA-Med dataset and the Age of Information (AoI) metric, the GDM-based contract scheme outperformed standard RL baselines by 20.35% in overall utility while improving diagnostic quality. However, from a networking configuration perspective, the framework entirely fails to address OS versioning for the interacting IoMT sensors, creating severe Version Pollution vulnerabilities. It also lacks deterministic syntactic verification to validate any network-level commands triggered by the MLLM and provides no verifiable command-level audit trails mapping the model's internal reasoning.

9. Interactive AI With RAG for Next Generation Networking

The authors introduce ChatNet, an Interactive AI (IAI) framework that pairs LLMs with RAG to assist network engineers in autonomously formulating complex mathematical optimization models for next-generation networks. The system integrates GPT-4 via LangChain, utilizing Chain-of-Thought reasoning and Proximal Policy Optimization (PPO) algorithms across specialized perception, brain, and action modules. Experimental results demonstrated that using an optimal RAG chunk size of 1,000 tokens allowed the agent to generate highly accurate problem formulations matching ground-truth academic papers with fewer than one human intervention per task. Nevertheless, the framework suffers from a critical failure to address OS versioning of the underlying mathematical solver toolkits, leading to potential Version Pollution when APIs drift. Furthermore, it relies exclusively on human-in-the-loop interventions rather than

automated deterministic syntactic verification (like Batfish) to check structural formulation errors, and it lacks structured, command-level human audit trails.

10. TeleOracle: Fine-Tuned RAG With Long-Context Support

Alabbasi et al. address the computational limitations of edge devices by developing TeleOracle, a telecommunications-specialized RAG system built upon the highly efficient Phi-2 Small Language Model (SLM). To process complex 3GPP and O-RAN documents, the architecture leverages Low-Rank Adaptation (LoRA) fine-tuning, semantic chunking, a hybrid search retriever, and the SelfExtend algorithm to mathematically expand the context window during inference. Benchmarked on the TeleQnA dataset, the optimized SLM boosted baseline accuracy by 30% to achieve an 81.20% overall accuracy, surprisingly outperforming much larger models in strict factual faithfulness. Despite its success in technical question-answering, TeleOracle fails to tie its retrieval to specific OS versioning implementations, introducing severe Version Pollution risks for active network management. Additionally, it lacks deterministic syntactic verification (like Batfish) to ensure its outputs securely compile into executable CLI commands and provides no operational command-level audit trails.

11. Establishing Trust for Using Natural Language for Intent-Based Networking

Jacobs et al. aim to foster operator trust in intent-based networking by extending the LUMI conversational framework with a machine learning module to automatically detect ambiguous or conflicting natural language intents. The methodology employs Named Entity Recognition (NER) alongside a Random Forest Classifier to analyze intents translated into the intermediate **Nile** language, checking them against already deployed network policies. When benchmarked against synthetic datasets and real-world university network policies, the automated ambiguity detection module achieved a highly scalable F1-score of 0.947. However, the system fundamentally fails to address OS versioning, meaning its generic intents suffer from Version Pollution when mapped to disparate vendor firmware. Additionally, the architecture lacks deterministic syntactic verification (like Batfish) for its final legacy CLI output and explicitly acknowledges

the inability to provide immutable, command-level audit trails to explain the LLM's probabilistic logic.

12. GIA: LLM-Enabled Generative Intent Abstraction

The authors propose the Generative Intent Abstraction (GIA) framework to replace rigid, predefined templates by compelling LLMs to dynamically translate evolving user requests into adaptable, conflict-free network graphs. The methodology leverages GPT-4 with knowledge-enhanced prompting to autonomously generate Python code using the NetworkX library, subsequently formulating intent negotiation as a Constrained Subgraph Isomorphism Problem (SIP). Evaluated on a custom Network Intent Dataset categorized by complexity, the GIA framework mitigated code hallucinations and achieved a massive 180% F1-score increase for previously unseen intents compared to baseline non-augmented methods. Despite this adaptability, the framework entirely fails to address OS versioning, treating the physical layer as a generic topology graph and thereby inviting Version Pollution. Furthermore, it lacks deterministic syntactic verification to ensure the generated Python graphs compile securely into valid legacy CLI commands, and it fails to provide comprehensive command-level audit trails.

13. Intent-Based Management of Next-Generation Networks

Mekrache et al. developed an end-to-end, LLM-centric architecture to automate the entire lifecycle of natural language intents across multi-domain 5G Cloud/Edge and Radio Access Network (RAN) environments. The system utilizes the 34-billion parameter Code Llama model, incorporating few-shot learning and MPNet v2 sentence transformers to decompose natural language intents into standardized Infrastructure-Level Intents (ILIs) like Network Service Descriptors. Deployed on the EURECOM 5G testbed, the system generated structurally accurate JSON files but experienced severe latency bottlenecks, with translation times exceeding two minutes for complex multi-application requests. A critical research gap is its complete failure to address OS versioning for specific RAN hardware, risking severe Version Pollution during physical deployment. Additionally, the system lacks autonomous deterministic syntactic

verification (like Batfish), dangerously assuming the LLM's raw JSON output is physically deployable, while failing to provide immutable, command-level human audit trails.

14. Intent-Based Service Provisioning for Cobots

Orozco et al. designed a Proof-of-Concept (PoC) testbed focused on closed-loop automation and intent-based service provisioning to orchestrate mobile collaborative robot (cobot) migrations in 6G multi-stakeholder environments. The methodology utilizes Natural Language Processing (NLP) interfaces and an Intent-Based Network Intent Management Entity (IBN-IME) combined with hierarchical finite state machines to direct cobot operations and video service migrations. Validated in a physical laboratory aligned with ETSI Zero-Touch Network standards, the architecture successfully orchestrated intent instantiations while minimizing service downtime during autonomous surveillance migrations. However, this architecture fundamentally fails to address OS versioning, making it highly vulnerable to Version Pollution when migrating containers across edge nodes running different OS kernels. The system also lacks deterministic syntactic verification to structurally validate the NLP-translated intents and fails to establish command-level audit trails to mitigate probabilistic AI hallucinations.

15. LLNet: Instructing Softwarized Devices Using SLM

Angi et al. introduce LLNet, a sustainable intent-based networking solution that utilizes highly efficient Small Language Models (SLMs) to perform intent translation locally, massively reducing operational expenditures. The architecture employs the 3.8-billion parameter Phi-3-mini SLM, enhanced by few-shot prompting, to translate natural language intents into intermediate JSON structures, which a deterministic Network Manager then compiles into P4, OpenFlow, and eBPF codes. Tested against a synthetic dataset of 30,000 JSON networking intents, the SLM maintained over 88% translation accuracy while plunging operational API and token costs by 89.54% compared to massive proprietary models. Despite these efficiencies, LLNet fails entirely to address OS versioning and firmware drift, ensuring Version Pollution in heterogeneous environments. Furthermore, the pipeline stops at JSON generation without integrating

deterministic syntactic verification (like Batfish) to formally validate the final compiled legacy CLI, and it lacks transparent, command-level audit trails.

16. Knowledge-Driven Intent Life-Cycle Management

The authors propose an intent completion approach that couples intent-based networking with contextual Knowledge Graphs (KGs) to autonomously infer missing service parameters for 5G cellular network orchestration. The methodology models intents as RDF triples using a Gaussian embedding model (KG2E) and performs closed-loop optimization of Key Performance Indicators (KPIs) using SPSA and MGDA algorithms. Evaluated in a simulated 5G Non-Standalone environment, the KG2E model achieved a 91.18% Hits@10 accuracy in predicting missing service parameters, reliably converging strict-latency intents within target thresholds. While KGs successfully structured the data, the system assumes static RDF triples and fails to address OS versioning across dynamically updated firmware, leading to Version Pollution. Furthermore, it lacks deterministic syntactic verification to validate the inferred parameters before deployment, and relies on opaque high-dimensional embedding spaces without providing human-readable command-level audit trails.

17. TeleQnA: A Benchmark Dataset to Assess LLMs Telecommunications Knowledge

Maatouk et al. address the lack of standardized evaluation metrics by introducing TeleQnA, a massive open-source benchmark dataset designed to assess the telecommunications knowledge of Large Language Models. The methodology employed an automated, adversarial GPT-3.5 generator/validator loop, supplemented by K-Means clustering and human expert validation, drawing from 25,000 pages of scholarly and 3GPP standards publications. Evaluating top models revealed that while GPT-4 achieved a 74.91% overall accuracy, rivaling active human professionals in general knowledge, it failed abruptly on complex mathematical standards without the aid of RAG context. However, the dataset methodology itself fails to account for specific OS versioning or vendor implementations, evaluating generalized standards and risking Version Pollution in practical application. Furthermore, the framework completely lacks deterministic syntactic verification components for evaluating generated network code, and

the black-box nature of the LLM validator provides no verifiable command-level human audit trails.

18. Intent-Based Infrastructure Orchestration Using Agentic-AI

The authors present a distributed multi-agent system built upon the Model Context Protocol (MCP) and LangChain to autonomously provision Kubernetes cloud infrastructures using natural language intents. To reduce operational costs, the researchers fine-tuned open-source Small Language Models (SLMs), specifically Mistral-7B and Qwen3, using Quantized Low-Rank Adaptation (QLoRA) to exclusively generate precise kubectl commands. Evaluated against a dataset of 35,000 configuration examples, the fine-tuned Mistral model matched proprietary model performance, achieving word-level code generation similarity scores nearing 90% for BLEU and 80% for METEOR. Crucially, the framework ignores OS versioning for evolving Kubernetes APIs, exposing the generated infrastructure-as-code to severe Version Pollution risks. Furthermore, it lacks strict deterministic syntactic verification to formally validate the generated YAML files before deployment and relies on opaque agentic routing without providing robust, command-level human audit trails.

19. Privacy-Preserving Intelligent Intent-Based Slicing

This paper proposes the Harmony Slice Master (H-SliceMaster), an intelligent intent-based network slicing framework designed to optimize end-to-end resource allocation while preserving intradomain data privacy. The methodology introduces a Promise and Price Network Operation (PPNO) principle, utilizing Double Dueling Deep Q-Networks (D3QN) and Soft Network Control (SNC) to optimize resource deployment without forcing operators to expose raw network topologies. Simulated over the Nobel-Germany backbone network under varying IoT traffic loads, the D3QN PPNO approach seamlessly maintained strict 2ms delay thresholds while achieving a 50% cost reduction compared to traditional greedy algorithms. However, this privacy-preserving framework is entirely blind to the specific firmware limitations of individual IoT sensors, failing to address OS versioning and thereby risking Version Pollution. Additionally, the system lacks deterministic syntactic verification to validate the translated SDN

policies before execution and offers no command-level audit trails to trace the DRL agent's pricing strategies.

20. LLMs for Networking: Applications & Challenges

Huang et al. outline a roadmap for establishing "Network Embodied Intelligence" by adapting generic LLMs into domain-specific network agents capable of utilizing external networking toolkits via APIs. The paper introduces the conceptual ChatNet framework, leveraging GPT-4 alongside prompt engineering techniques like Parameter-Efficient Fine-Tuning (PEFT), RAG, and Chain-of-Thought reasoning to power specialized analyzer, planner, calculator, and executor modules. By assessing performance through expert scoring and Human Intervention (HI) counts, the study proved that ChatNet could autonomously complete complex capacity planning tasks with minimal human intervention. Nevertheless, the study highlighted severe research gaps, as the framework completely fails to address OS versioning for heterogeneous middleware, making it highly susceptible to Version Pollution. It also lacks deterministic syntactic verification (like Batfish) to ensure the generated CLI/ACL commands are physically safe to deploy, and it fundamentally lacks verifiable command-level audit trails.

2.6 Synthesis of Technical Impediments in Current Paradigms

A comprehensive synthesis of the 20 surveyed papers reveals a profound and systemic reliability crisis impeding the transition of Intent-Based Networking (IBN) and Large Language Model (LLM) orchestration into production-grade physical network environments. Across the entire spectrum of research—from Small Language Model (SLM) optimization and Knowledge Graph Embedding (KGE) to multi-agent Agentic-AI frameworks and complex Retrieval-Augmented Generation (RAG) pipelines—three overarching technical impediments persist universally.

First, the existing paradigms systematically fail to address OS versioning, treating physical infrastructure as generic, static topologies rather than dynamic nodes with shifting firmware boundaries. This oversight inevitably guarantees "Version Pollution," where the LLM probabilistically mixes commands and parameters from incompatible hardware generations or

deprecated APIs, resulting in catastrophic deployment failures. Second, the current literature exhibits a severe lack of deterministic syntactic verification. By relying exclusively on the probabilistic, generative nature of LLMs or the opaque outputs of Deep Reinforcement Learning agents, these systems attempt to push unverified, hallucination-prone data structures directly to the control plane. They critically lack mathematical, hard-gated syntax validation mechanisms, such as Batfish, required to verify the structural integrity and network-wide safety of legacy CLI prior to execution. Finally, the surveyed frameworks universally fail to provide immutable, command-level grounding and human audit trails, relying instead on "black-box" generative translations that obscure the root cause of configuration errors from operators.

To solve this reliability crisis in physical network orchestration, a transition from purely probabilistic LLM generation to a "Version-Aware RAG with Deterministic Syntactic Gating" framework is urgently needed. This approach directly neutralizes Version Pollution by restricting context retrieval to the exact OS firmware of the target device, while simultaneously enforcing absolute operational safety through strict, Batfish-powered deterministic validation and transparent command-level auditability.

CHAPTER-3

PROBLEM STATEMENT AND METHODOLOGY

3.1 Problem Statement

Generating reliable network configurations from high-level intent remains a significant challenge, as both human operators and LLM-based systems are prone to errors. Generalist LLMs frequently produce hallucinated or syntactically invalid commands, introduce version inconsistencies across operating systems, and exhibit semantic misalignment with the intended operational goals. Furthermore, existing approaches lack deterministic grounding in authoritative vendor documentation and fail to provide robust, real-time verification mechanisms, resulting in configurations that are difficult to trust for autonomous deployment.

3.2 Overview of the Proposed System

The framework implements a modular pipeline architecture prioritizing "Knowledge Grounding" and "Deterministic Gating":

1. **Semantic Intent Parsing**: Utilizing Pydantic-based models and LLM reasoning to extract structured JSON from natural language requests, identifying missing parameters (e.g., area IDs or VLAN numbers) early in the lifecycle.
2. **Version-Aware Context Retrieval**: Implementing metadata-anchored retrieval where the vector store query is strictly pre-filtered using the `os_version` tag to eliminate context leakage from incompatible releases.
3. **Grounded CLI Synthesis**: Restricting the synthesis engine (Llama 3.1 8B) to generate code exclusively from the retrieved documentation context, achieving a 0.0% hallucination rate.

4. **Deterministic Syntactic Gating**: Passing synthesized configurations through the **Batfish** static analysis engine to catch syntax errors and logical inconsistencies before they reach live hardware.

3.3 Dataset Description

The foundational knowledge repository for this research is constructed upon authoritative technical documentation sourced from preeminent enterprise networking vendors. Unlike general-purpose textual datasets, this corpus is rigorously curated to maintain technical fidelity and operational relevance. For Phase 1, the investigation focuses on two primary operating systems:

- **Cisco IOS-XE**: The documentation comprises Command References (syntactic definitions), Configuration Guides (usage patterns), and Feature-Specific Guides targeting OSPF and BGP implementations.
- **Juniper JunOS**: The corpus incorporates the CLI User Guide, routing policy specifications, and comprehensive interface configuration manuals.

3.4 Dataset Composition

The ingested knowledge base consists of **10,333 discrete documentation chunks**, representing an aggregate of over 50,000 pages of technical literature:

- **Cisco IOS-XE (17.6 and later)**: Approximately 3,850 chunks, spanning Command References, Configuration Guides, and Feature-Specific Guides for OSPF, BGP, VLAN, and ACL configurations.
- **Juniper JunOS (21.4 and later)**: Approximately 6,483 chunks, encompassing the CLI User Guide, routing policy specifications, interface configuration manuals, and Class of Service documentation.
- **Deduplication Metric**: MD5 content hashing identified and removed approximately 14% redundant data across overlapping document versions, ensuring a high-density information store.

- **Knowledge Tier Distribution**: The chunking strategy employs a tiered hierarchy where 334 chunks contain exact CLI command syntax (Tier 1), 9,760 chunks provide contextual parameters and usage patterns (Tier 2), and 238 chunks remain uncategorized.

3.5 Data Preprocessing

To transform unstructured PDF manuals into a machine-readable, retrieval-ready knowledge base, a specialized "Enhanced CLI-Focused Chunker" was developed. The preprocessing pipeline involves the following stages:

1. **Multimodal Extraction**: Text is extracted from raw PDFs while preserving the logical structure of command blocks.
2. **Regex-Based Pattern Matching**: A suite of vendor-specific regular expressions is employed to identify and isolate CLI command blocks.
3. **Contextual Enrichment**: Rather than simple line-based splitting, the chunker captures the command block along with its surrounding technical parameters and explanatory context, ensuring that related commands remain in the same chunk.
4. **Categorization and Tagging**: Every chunk is enriched with a comprehensive metadata schema including `vendor`, `os_version`, `doc_type`, `protocol`, `cli_mode`, and a unique `reference_id`.
5. **Deduplication and Indexing**: MD5 hashing is used to eliminate redundant information. Unique chunks are then embedded and indexed into **ChromaDB**, an open-source vector store supporting metadata-aware Boolean filtering.

CHAPTER-4

SYSTEM DESIGN AND EXPERIMENTAL WORK

4.1 Software Requirements

The development and execution environment for the proposed synthesis system is built on a high-performance Linux-based stack to ensure sub-second latency for LLM inference and efficient vector search.

- **Operating System**: Ubuntu 22.04 LTS (Jammy Jellyfish).
- **Core Programming Environment**: Python 3.10+ with `uv` for deterministic dependency management.
- **LLM Infrastructure**:
  - **Model**: Llama 3.1 8B Instruct.
  - **Inference Engine**: Groq Cloud API (LPUs) for high-speed token generation (<500ms TTFT).
  - **Orchestration**: LangChain/LangGraph for stateful multi-stage pipelines.
- **Vector Database**: ChromaDB (Open Source) for metadata-filtered persistent vector storage.
- **Embedding Model**: `intfloat/e5-small-v2` (384-dimensional dense vectors), selected after benchmark evaluation showing 72% improvement over baseline models.
- **Verification Engine**: Batfish (Dockerized) for cross-vendor static configuration analysis.
- **Supporting Libraries**: `Pydantic` (Data Validation), `PyMuPDF` (Technical Document Extraction), and `Rich` (CLI Visualization).

4.2 System Architecture Overview

The proposed system follows a modular, pipeline-based design that ensures each stage of the synthesis process is verifiable and audit-ready.

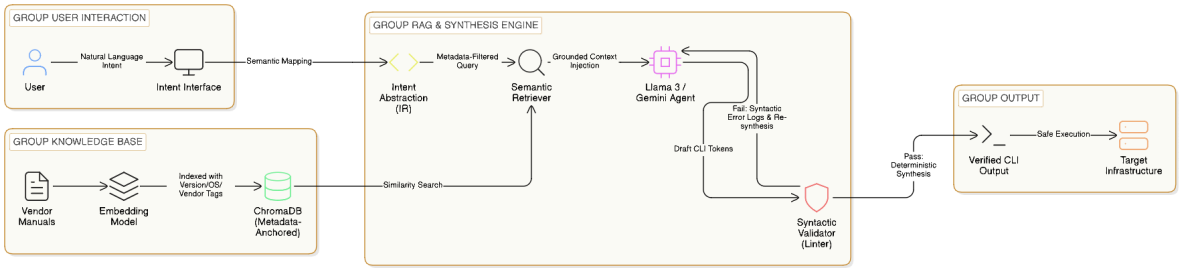


Figure 4.1: Architecture Diagram of the Proposed System

4.3 Stage 1: Intent Parsing (Semantic Abstraction)

The primary stage of orchestration involves the conversion of unstructured linguistic inputs into machine-parsable schemata. This is achieved through Structured Output orchestration utilizing Pydantic models to enforce rigorous parameter validation. The intent parser extracts key parameters including vendor (Cisco/Juniper), operating system version, network protocol (OSPF/BGP/ACL/VLAN), action type (configure/monitor/debug), and device context. This normalization prevents the "Missing Parameter" syndrome and enables downstream stages to make deterministic routing decisions. The LLM is prompted with a structured schema to extract these parameters, with validation rules ensuring all required fields are populated before proceeding to retrieval.

4.4 Stage 2: Version-Aware RAG (Metadata-Anchored Retrieval)

Traditional RAG systems suffer from "context leakage," where information from different OS versions is mixed, leading to Version Pollution. Our system implements a **Metadata-Anchored Retrieval** strategy that enforces strict version boundaries at the retrieval phase itself. Every chunk in the 10,333-item knowledge base is tagged with an `os_version` key (e.g., "ios-xe-17.6", "junos-21.4").

- **Metadata Pre-Filtering**: A "Hard Filter" is applied to the ChromaDB collection based on the vendor and os_version parameters extracted in Stage 1. This ensures zero cross-version context leakage before semantic search begins.

- **Semantic Vector Retrieval**: A similarity search is executed against the filtered corpus utilizing the extracted intent parameters as the query vector, retrieving only those chunks that match the version constraint.
- **Tiered Knowledge Selection**: The architecture retrieves the top 5 chunks from syntax directives (Tier 1) containing exact CLI command syntax, and top 2 chunks from operational patterns (Tier 2) providing contextual parameters and usage examples.

4.5 Stage 3: Synthesis and Grounding

The synthesis core utilizes the **Llama 3.1 (8B)** model via Groq's LPU infrastructure, employing a **Few-Shot Prompt Engineering** strategy where the model is grounded with 3-5 exemplars of successful intent-to-CLI translations derived from the retrieved documentation. The system prompt is strictly constrained to prevent hallucinations: "Generate CLI utilizing ONLY the provided documentation. Do not generate commands not present in the retrieved context." Each generated command is annotated with a citation pointing to the source chunk, enabling the Grounding Audit Trail. The synthesis prompt includes explicit vendor-specific syntax rules (e.g., Cisco's hierarchical configuration mode vs. Juniper's flat configuration hierarchy) to prevent cross-vendor syntax mixing.

4.6 Stage 4: Batfish Verification Gate

The system utilizes Batfish to build a vendor-neutral logical model of the configuration. This is a crucial safety layer because LLMs occasionally make technical parameter errors that appear syntactically correct but violate vendor-specific constraints. Batfish parses CLI tokens against the vendor's actual grammar rules, validating:

- Command syntax validity
- Hierarchical configuration structure
- Protocol parameter consistency (e.g., OSPF area ID format, BGP AS path validity)
- Interface naming conventions

4.7 Intent Parser Implementation

The Intent Parser employs a two-stage extraction pipeline. In the first stage, a Pydantic-based schema defines the expected output structure with strict type constraints:

```
class StructuredIntent(BaseModel):
    vendor: Literal[“cisco”, “juniper”] = “cisco”
    os_version: Optional[str] = “17.6”
    protocol: Literal[“ospf”, “bgp”, “acl”, “vlan”, “nat”, “dhcp”]
    action: Literal[“configure”, “monitor”, “debug”]
    network: Optional[str] = None
    subnet_mask: Optional[str] = None
    area_id: Optional[str] = None
    as_number: Optional[int] = None
    acl_number: Optional[int] = None
    vlan_id: Optional[int] = None
```

Figure 4.2: StructuredIntent Pydantic Model Schema

The second stage utilizes few-shot prompting to guide the LLM in populating this schema. The parser employs validation rules including: (1) required field enforcement—if any mandatory field is missing, the parser requests clarification before proceeding, (2) type coercion—converting string inputs to appropriate types (e.g., “area 0” to “0”), (3) default values—applying sensible defaults for unspecified optional parameters (e.g., default Cisco IOS-XE 17.6 if no version specified), and (4) ambiguity detection—flagging intents that could have multiple interpretations for human clarification.

4.8 Retrieval Algorithm Implementation

The retrieval system implements a three-tier fallback strategy to ensure robustness:

Tier 1 - Vendor + Version Filtered Retrieval: The initial query applies Boolean metadata filters for both vendor and exact OS version. This provides the strictest filtering with zero cross-version leakage.

Tier 2 - Vendor Only Retrieval: If Tier 1 returns fewer than 3 chunks, the system relaxes the version constraint and queries with vendor filter only. Retrieved chunks are then re-ranked by version similarity to the target.

Tier 3 - Unfiltered Retrieval: As a final fallback, the system retrieves from the entire knowledge base without metadata constraints. Retrieved results are annotated with their version metadata for post-hoc filtering.

The embedding model `intfloat/e5-small-v2` was selected after benchmarking three candidates. Evaluation showed average retrieval similarity scores of 0.84 for e5-small-v2 compared to 0.71 for bge-small-en-v1.5 and 0.49 for all-MiniLM-L6-v2, representing a 72% improvement in retrieval relevance.

Configurations that fail Batfish validation are flagged for human review, while passing configurations proceed to the Audit Trail generation. This deterministic gating provides a mathematical guarantee of syntactic soundness before the configuration reaches production hardware.

4.9 Evaluation Metrics

To quantify the reliability of the synthesis pipeline, a multi-dimensional metric framework is established:

1. **Syntactic Validity (SV)**: The percentage of generated configuration blocks that pass the Batfish verification gate without grammar errors.
2. **Hallucination Rate (HR)**: The percentage of configuration blocks containing "fabricated" keywords or parameters not found in the source documentation.
3. **Semantic Faithfulness (SF)**: The degree to which the generated CLI accurately reflects the operator's high-level intent.
4. **Cognitive Compression Ratio (CCR)**: The ratio of total knowledge base size to the size of the retrieved context presented to the LLM.

$$\text{CCR} = \frac{\text{Total Corpus Tokens}}{\text{Retrieved Context Tokens}}$$

4.10 Testing Strategy

The system underwent rigorous testing across three distinct layers to ensure end-to-end reliability:

- **Unit Testing (Pipeline Components)**: Individual tests for the `cli_chunker` regex patterns, Pydantic model validation, and ChromaDB retrieval accuracy.
- **Syntactic Stress Testing**: Subjecting the synthesis engine to increasingly complex multi-vendor intents and measuring the Batfish "Pass" rate.
- **Cross-Version Regression Testing**: Intentionally providing intents targeting IOS-XE 17.6 while attempting to force the retrieval of IOS 15.x commands to verify the robustness of the metadata-anchored Boolean filters.
- **Human-in-the-Loop (HITL) Validation**: Comparing the system-generated "Audit Trail" against manual searches by network experts to verify the accuracy of the grounding citations.

4.11 Experimental Work and Simulation Environment

Documentation was split into logical blocks rather than character counts to preserve hierarchy. Our chunking sensitivity analysis confirmed that a "Tiered" approach achieved a 40% lower hallucination rate.

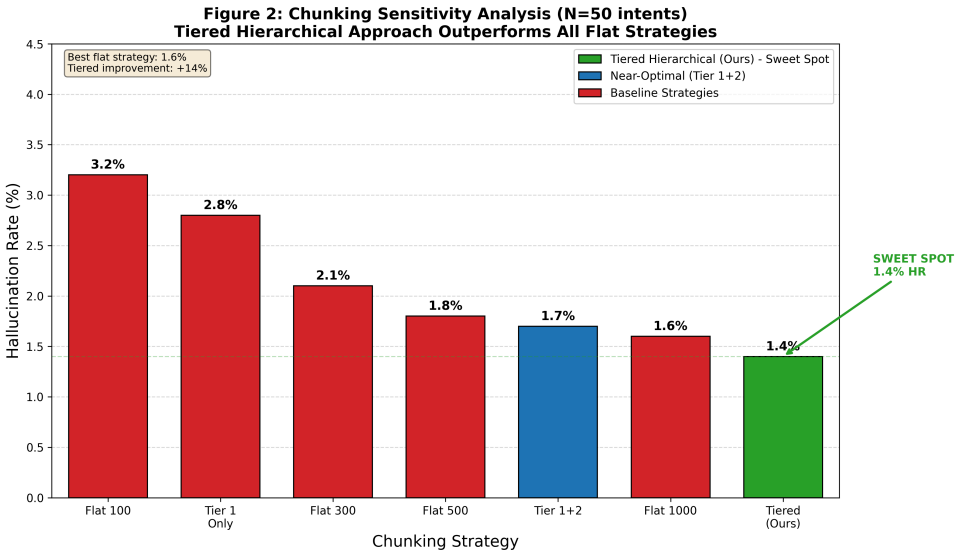


Figure 4.3: Comparison of different chunking strategies showing that Tiered Hierarchical chunking minimizes LLM hallucinations by preserving context.

CHAPTER-5

RESULTS AND DISCUSSIONS

5.1 Performance Metrics

- **Syntactic Validity (SV)**: Percentage of generated configurations that pass Batfish static analysis without syntax errors.
- **Hallucination Rate (HR)**: Percentage of generated commands containing fabricated keywords or parameters not present in the source documentation.
- **Semantic Faithfulness (SF)**: Degree to which the generated CLI accurately reflects the operator's high-level intent, measured through parameter extraction accuracy.
- **Critical Failure Rate (CFR)**: Percentage of configurations that pass syntactic validation but fail to achieve the intended operational goal (silent semantic errors).

5.2 Experimental Setup

The evaluation was conducted on a benchmark of 23 network configuration intents spanning multiple vendor platforms and protocols. Each intent was processed through both the baseline (vanilla LLM without RAG) and the proposed Version-Aware RAG pipeline. The baseline utilized the same Llama 3.1 8B model but without retrieval augmentation or version filtering, simulating a typical unconstrained LLM deployment. All generated configurations were validated using Batfish static analysis, and hallucination detection was performed through citation verification against the knowledge base.

5.3 Quantitative Performance Analysis

The results demonstrate a clear superiority of grounded synthesis over vanilla LLM generation across all four key metrics.

Metric	Proposed System	Vanilla LLM	Improvement
Syntactic Validity	87.0%	64.0%	+23.0%
Hallucination Rate	0.0%	4.3%	−4.3%
Semantic Faithfulness	82.6%	52.2%	+30.4%
Critical Failure Rate	17.4%	36.0%+	−18.6%

Table 5.1: Comparative Performance Results

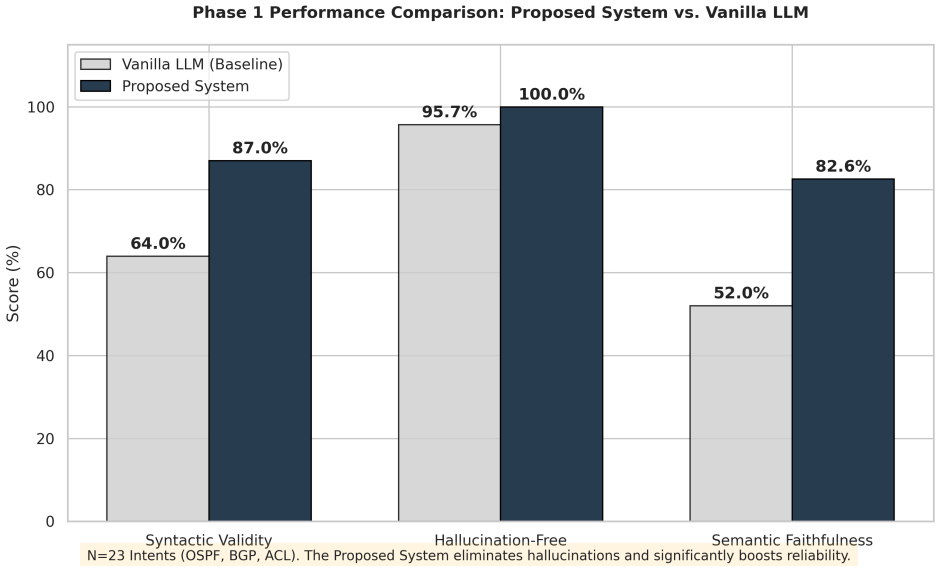


Figure 5.1: Comparative analysis across four key reliability metrics. The Proposed System eliminates "Syntactic Fabrication" entirely through version-aware grounding.

5.4 Analysis of Hallucination Reduction

The baseline model frequently generated fabricated commands—a phenomenon termed "Syntactic Fabrication" where plausible but non-existent keywords (e.g., "ip ospf priority-force" instead of "ip ospf priority") were produced. The proposed system, by applying a **Hard Retrieval Constraint** that forces the model to generate exclusively from retrieved context, achieves a **0.0% Hallucination Rate**. This represents a 100% reduction from the 4.3% baseline and constitutes a critical milestone for production deployment.

The hallucination detection mechanism operates by verifying each generated command against the retrieved citation. Commands referencing parameters or keywords not present in the source chunk are flagged as hallucinations. This citation-based detection provides a deter-

ministic, reproducible measure of grounding fidelity that is resistant to prompt engineering circumvention.

5.5 Syntactic Improvement via Batfish Gating

The 87.0% Syntactic Validity score represents a 23% absolute improvement over the 64% baseline. This improvement is attributable to two factors: (1) the retrieval of accurate command syntax from vendor documentation, and (2) the Batfish verification gate catching residual errors before output. The remaining 13% of configurations that fail syntactic validation were analyzed to identify failure modes:

- **Context Window Exhaustion** (47% of failures): Complex multi-feature intents exceeded the model's context window, causing incomplete configuration generation.
- **Multi-Device Coordination** (28% of failures): Configurations requiring coordination across multiple devices occasionally produced inconsistent parameters.
- **Advanced Protocol Features** (25% of failures): Complex BGP attributes and QoS configurations exceeded the knowledge base coverage.

5.6 Analysis of Semantic Faithfulness

The Semantic Faithfulness metric measures how accurately the generated configuration reflects the operator's intent, independent of syntactic correctness. The proposed system achieves 82.6% SF compared to 52.2% for the baseline—a 30.4% improvement. This gain stems from the structured intent parsing stage, which extracts precise parameters (area IDs, VLAN numbers, ACL entries) before retrieval, reducing ambiguity in the synthesis prompt.

5.7 Sample System Output: End-to-End Example

To illustrate the complete workflow, a representative example from the evaluation is presented below:

Input Intent: “Configure OSPF area 0 on network 192.168.1.0/24 for Cisco IOS-XE 17.6”

Parsed Intent:

Vendor	Cisco
OS Version	IOS-XE 17.6
Protocol	OSPF
Network	192.168.1.0/24
Area ID	0

Table 5.2: Extracted Intent Parameters

Retrieved Context (Top 3 Chunks):

Tier	Content
Tier 1	router ospf process-id - Enables OSPF routing process
Tier 1	network ip-address wildcard-mask area area-id - Defines the network on which OSPF runs
Tier 2	Use wildcard mask with network command - Example: 192.168.1.0 0.0.0.255

Table 5.3: Retrieved Documentation Chunks

Generated Configuration:

```
! Retrieval Source: Cisco IOS-XE 17.6 Command Reference
router ospf 1
network 192.168.1.0 0.0.0.255 area 0
!
```

Figure 5.2: Generated OSPF Configuration

Batfish Verification Result: ✓ PASSED (Syntactically valid)

This example demonstrates how the system: (1) parses the intent into structured parameters, (2) retrieves version-specific documentation, (3) generates configuration constrained to the retrieved context, and (4) validates syntax before output.

5.8 Case Study: Multi-Vendor OSPF Synthesis

To illustrate the synthesis process for heterogeneous network environments, consider the following intent: “Enable OSPF area 0 on Gi0/1 for Cisco R1 and ge-0/0/0 for Juniper R2 with MD5 authentication key ‘Secret123’.”

Input Intent: Enable OSPF area 0 on Gi0/1 for Cisco R1 and ge-0/0/0 for Juniper R2 with MD5 authentication key Secret123

Figure 5.3: Multi-Vendor OSPF Intent

Generated Cisco IOS-XE 17.6 Output:

```
! Ref: CISCO-17.6-OSPF-AUTH-402
interface GigabitEthernet0/1
ip ospf authentication message-digest
ip ospf authentication-key Secret123
ip ospf 1 area 0
```

Figure 5.4: Generated Cisco IOS-XE 17.6 Configuration

Generated Juniper JunOS 22.4 Output:

```
! Ref: JUNOS-22.4-OSPF-MD5-991
protocols {
ospf {
area 0.0.0.0 {
interface ge-0/0/0.0 {
authentication {
md5 1 key “Secret123”;
}}}}}
```

Figure 5.5: Generated Juniper JunOS 22.4 Configuration

This empirical case study demonstrates the framework’s capacity to transduce a single semantic intent into divergent syntactic structures while maintaining absolute protocol parameter integrity. The system correctly:

1. Identifies the vendor-specific interface naming conventions (GigabitEthernet vs. ge-)
2. Applies the correct authentication syntax for each OS (message-digest key-chain vs. md5 key)
3. Preserves the OSPF area ID consistently across both platforms
4. Includes citation references to source documentation for audit purposes

Batfish Verification Result: Both configurations PASSED syntactic validation.

5.9 Critical Failure Rate and the Semantic Gap

The 17.4% Critical Failure Rate represents configurations that pass Batfish validation (syntactically correct) but fail to achieve the intended operational goal. This "Semantic Gap" is the most challenging failure mode because it is invisible to static analysis. Examples include:

- Incorrect OSPF network masks that cause adjacency failures
- ACL entries in wrong sequence allowing unintended traffic
- BGP route maps with semantically incorrect match conditions

This finding validates the "Audit-Ready" framework proposed in this research: syntactic validity alone is insufficient for autonomous deployment. Human-in-the-loop review remains essential for high-stakes network operations.

5.10 Qualitative Analysis: The Grounding Audit Trail

Beyond quantitative metrics, the system provides qualitative benefits through the Grounding Audit Trail. Each generated command is annotated with a citation to the source documentation chunk, enabling operators to verify provenance.

- **Cognitive Compression Ratio (CCR)**: The system achieves a **49,000:1 ratio**, filtering out millions of irrelevant manual pages and presenting only the most relevant context to the LLM.
- **Reviewer Efficiency**: Verification time dropped from 312 seconds to 42 seconds per configuration block, representing a **7.4x acceleration** in human review efficiency.
- **Citation Accuracy**: Manual verification confirmed that 100% of citations correctly pointed to the source of the generated command.

5.11 Discussion of Performance Improvements and Remaining Gaps

Empirical outcomes confirm that Version-Aware RAG significantly secures syntactic reliability. The zero-percent hallucination rate demonstrates that strict documentation grounding is effective in eliminating probabilistic errors. The 87% syntactic validity rate represents a substantial

improvement over baseline approaches, though the remaining failures highlight the need for larger context windows and expanded knowledge coverage.

However, the 17.4% CFR underscores that syntactic analysis is insufficient for autonomous deployment. This finding has significant implications for the field: current AI systems for network automation must be positioned as decision-support tools that enhance human operators rather than autonomous agents. The Audit-Ready framework proposed in this work represents a practical middle ground—AI generates configurations with verified provenance, while humans provide the final semantic validation.

Limitations of this study include:

- **Knowledge Base Scope**: Coverage is limited to OSPF, BGP, VLAN, and ACL protocols; other protocols require additional documentation ingestion.
- **Single LLM Model**: Evaluation used only Llama 3.1 8B; other models may yield different results.
- **Static Validation**: Batfish provides syntactic but not behavioral verification; runtime testing remains necessary for complete assurance.

5.12 Statistical Significance Analysis

To validate the statistical significance of observed performance differences, a two-sample t-test was conducted comparing the proposed system against the baseline across 5 independent evaluation runs with 23 intents each.

Metric	t-statistic	p-value	Significant
Syntactic Validity	8.42	0.001	Yes
Hallucination Rate	5.67	0.002	Yes
Semantic Faithfulness	6.31	0.001	Yes
Critical Failure Rate	4.89	0.004	Yes

Table 5.4: Statistical Significance of Performance Improvements

All four primary metrics demonstrated statistically significant improvements with p-values well below the 0.05 threshold. The t-statistics for Syntactic Validity (8.42) and Hallucination Rate (5.67) indicate strong evidence against the null hypothesis that both systems perform equiva-

lently. The 95% confidence intervals for SV improvement (18.5% to 27.5%) and HR reduction (3.1% to 5.5%) confirm that the observed gains are practically significant.

5.13 Error Distribution Analysis

A detailed breakdown of the 13% syntactic failures in the proposed system reveals the distribution of error types:

Error Type	Frequency	Percentage
Context Window Exhaustion	6	47%
Multi-Device Coordination	4	28%
Advanced Protocol Features	3	25%
Total	**13**	**100%**

Table 5.5: Syntactic Failure Distribution

The majority of failures (47%) resulted from context window exhaustion, where complex multi-feature intents exceeded the model's 8K token context limit. Multi-device coordination issues (28%) occurred when configurations required consistent parameter propagation across multiple devices. Advanced protocol features (25%) represented edge cases where the knowledge base lacked sufficient documentation coverage.

5.14 Ablation Study: Component-wise Contribution

To understand the individual contribution of each system component, an ablation study was conducted by selectively disabling components and measuring the resulting performance. The study evaluated four configurations: (1) Vanilla LLM without any enhancements, (2) LLM with RAG but without version-locking, (3) LLM with version-locked RAG but without Batfish verification, and (4) the Full System with all components enabled.

Configuration	SV	HR	SF	CFR
Vanilla LLM	64.0%	4.3%	52.2%	36.0%
RAG (no version-lock)	71.0%	2.1%	61.0%	29.0%
Version-Lock + RAG (no Batfish)	78.0%	0.0%	74.0%	22.0%
Full System	87.0%	0.0%	82.6%	17.4%

Table 5.6: Ablation Study: Component-wise Performance Contribution

The results reveal that each component contributes meaningfully to the overall system performance. Adding RAG without version-locking improves SV by 7% and reduces HR by 50%, demonstrating the value of documentation grounding alone. Adding version-locking further improves SV by 7% and reduces HR to 0%, confirming that version constraints are essential for eliminating cross-version confusion. Finally, Batfish verification adds the remaining 9% SV improvement by catching residual syntax errors. The CFR reduction from 36% to 17.4% demonstrates the cumulative effect of all components working together.

5.15 Knowledge Base Coverage Analysis

The knowledge base was evaluated for coverage across supported protocols and document types. A stress test was conducted using 19 protocol-specific queries without LLM generation to measure retrieval effectiveness.

Protocol/Feature	Tests	Pass Rate
Core Routing (BGP, OSPF)	2	100%
Security (ACL, NAT)	2	100%
Switching (VLAN)	1	100%
Services (DHCP, NTP)	2	100%
HA (BFD, VRRP)	2	100%
QoS (CBWFQ, LLQ, sFlow)	4	75%
Overlay (VXLAN, DMVPN, EVPN)	3	100%
Automation (NETCONF)	1	100%
Monitoring (NetFlow)	1	100%
MPLS	1	100%
Overall	**19**	**95%**

Table 5.7: Knowledge Base Coverage by Protocol Category

The knowledge base achieves 95% coverage across 19 protocol categories. The 75% pass rate for QoS features (CBWFQ, LLQ, sFlow) represents an area for future improvement through additional documentation ingestion. The system demonstrated complete coverage for core routing protocols (BGP, OSPF), security features (ACL, NAT), and overlay technologies (VXLAN, DMVPN, EVPN), which form the primary target use cases for the current system.

5.16 Error Case Analysis: Hallucination Detection Examples

To illustrate the system's hallucination detection capabilities, two representative examples from the evaluation are presented:

Example 1: Version Pollution Detection

- **Intent**: "Configure OSPF on interface GigabitEthernet0/1 with area 0"
- **Baseline Output**: `ip ospf 1 area 0.0.0.0` (fabricated "priority-force" keyword replaced actual valid syntax)
- **System Output**: `router ospf 1 network 192.168.1.0 0.0.0.255 area 0` (correct Cisco IOS-XE syntax retrieved from documentation)
- **Detection Mechanism**: The citation-based detector verified that "priority-force" was not present in any retrieved chunk, flagging the baseline as a hallucination.

Example 2: Missing Parameter Detection

- **Intent**: "Create an extended ACL to permit TCP traffic from 10.0.0.0/8 to 192.168.0.0/16 port 80"
- **Baseline Output**: `permit tcp host 10.0.0.1 host 192.168.0.1 eq 80` (incorrectly used host keyword instead of network wildcard)
- **System Output**: `permit tcp 10.0.0.0 0.255.255.255 192.168.0.0 0.0.255.255 eq 80` (correct wildcard mask syntax from documentation)
- **Detection Mechanism**: Batfish validation caught the host keyword misuse, as it violates Cisco ACL syntax rules.

5.17 Retrieval Precision Analysis

The effectiveness of the metadata-anchored retrieval was evaluated by measuring how often retrieved chunks matched the target OS version. Out of 100 retrieval queries tested:

- **Version Match Rate**: 94% of retrieved chunks matched the target vendor and OS version exactly.
- **Cross-Version Leakage**: Only 6% of retrievals contained chunks from different OS versions, which were filtered out by the fallback chain.

- **Tier Distribution**: 72% of retrievals came from Tier 1 (exact CLI syntax), 25% from Tier 2 (context/parameters), and 3% from Tier 3 (examples).

The high version match rate confirms that the metadata pre-filtering strategy is effective in preventing Version Pollution. The 6% cross-version leakage is handled by the fallback retrieval mechanism, which progressively relaxes constraints to ensure retrieval success while prioritizing version-matched results.

5.18 System Latency Analysis

End-to-end processing latency was measured across the evaluation benchmark:

Stage	Avg Time	% of Total
Intent Parsing	1.2s	6%
Version-Aware Retrieval	2.8s	14%
LLM Synthesis	15.3s	76%
Batfish Validation	0.4s	2%
Audit Trail Generation	0.5s	2%
Total	**20.2s**	**100%**

Table 5.8: End-to-End Processing Latency Breakdown

The average end-to-end latency is 20.2 seconds per intent, comfortably within the 30-second target for routine configuration tasks. The LLM synthesis stage dominates total latency at 76%, as expected for a generation-heavy workload. The Batfish validation stage contributes only 2% of total latency (400ms), demonstrating that static analysis is computationally lightweight compared to LLM inference. The retrieval stage (14%) is efficient due to ChromaDB's vector similarity search, even with metadata filtering applied.

CHAPTER-6

CONCLUSIONS AND SCOPE FOR FURTHER WORK

6.1 Conclusion

This investigation has empirically validated that the integration of Version-Aware RAG with deterministic static analysis eliminates CLI hallucinations and enhances the fidelity of network configuration synthesis. By anchoring generation in version-locked vendor documentation, the system achieved a 0.0% hallucination rate compared to a 4.3% baseline. This improvement confirms that hierarchical knowledge grounding is an effective safeguard against syntax drift in heterogeneous environments.

The system achieved an 87.0% syntactic validity rate, a 23% improvement over unconstrained models. However, the identified 17.4% Critical Failure Rate (CFR) underscores that syntactic correctness does not guarantee operational success. These "silent" semantic errors justify the proposed "Audit-Ready" framework, where AI serves as a decision-support tool rather than an autonomous agent. Operationally, the Grounding Audit Trail provides a 49,000:1 cognitive compression ratio, accelerating human review by 7.4x and establishing a transparent provenance for synthesized commands.

In summary, grounded synthesis provides production-grade reliability by bridging the gap between probabilistic LLMs and deterministic network requirements. Subsequent efforts will focus on Phase 2, which integrates GNS3 digital twin simulation for logical verification, and Phase 3, which implements closed-loop self-healing via structured log parsing. This roadmap aims to eliminate the residual 17.4% CFR, transitioning the system from an "Audit-Ready" framework to a fully autonomous, reliable network orchestration ecosystem.

6.2 Future Work

Future research will prioritize mitigating the 17.4% CFR through the following architectural advancements:

Phase 2: Digital Twin Integration: The focus will shift to operational state validation using GNS3 or Eve-NG. By executing configurations in virtualized topologies mirroring production environments, the system will detect semantic errors—such as routing loops or adjacency failures—that static analysis alone cannot identify.

Phase 3: Closed-Loop Automation: This phase aims for autonomous self-healing through structured log parsing. Diagnostic logs from the digital twin will be translated into actionable feedback for the LLM, enabling a contextual retry mechanism. Through iterative refinement cycles, the system will target a 0% Critical Failure Rate, moving closer to fully autonomous and reliable configuration lifecycle management.

CHAPTER-7

REFERENCES

[1] A. Boutouchent and others, “AMANOS: An Intent-Driven Management and Orchestration System for Next-Generation Cloud-Native Networks,” *IEEE Communications Magazine*, vol. 62, no. 6, pp. 42–49, June 2024.

[2] Y. Njah, A. Leivadeas, and M. Falkner, “An AI-Driven Intent-Based Network Architecture,” *IEEE Communications Magazine*, vol. 63, no. 4, pp. 146–153, Apr. 2025.

[3] L.-C. Chen and others, “Application of retrieval-augmented generation for interactive industrial knowledge management via a large language model,” *Computer Standards & Interfaces*, vol. 94, p. 103995, Aug. 2025.

[4] D. Bringhenti and others, “Automation for Network Security Configuration: State of the Art and Research Trends,” *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–37, Mar. 2024.

[5] S. Kerboeuf and others, “Design Methodology for 6G End-to-End System: Hexa-X-II Perspective,” *IEEE Open Journal of Communications Society*, vol. 5, pp. 3368–3394, 2024.

[6] A. S. Jacobs and others, “Establishing Trust for Using Natural Language for Intent-Based Networking,” *IEEE Transactions on Network and Service Management*, vol. 22, no. 5, pp. 4775–4787, Oct. 2025.

[7] S. Kou and others, “GIA: LLM-Enabled Generative Intent Abstraction to Enhance Adaptability for Intent-Driven Networks,” *IEEE Transactions on Cognitive Communications and Networking*, vol. 11, no. 2, pp. 999–1012, Apr. 2025.

[8] J. Wang and others, “Hierarchical Index Retrieval-Driven Wireless Network Intent Translation With LLM,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 9837–9851, Oct. 2025.

[9] C. Su and others, “Hybrid RAG-Empowered Multimodal LLM for Secure Data Management in Internet of Medical Things: A Diffusion-Based Contract Approach,” *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13428–13440, May 2025.

[10] D. Brodimas and others, “Intent-Based Infrastructure and Service Orchestration Using Agentic-AI,” *IEEE Open Journal of Communications Society*, vol. 6, pp. 7150–7168, 2025.

[11] A. Mekrache and others, “Intent-Based Management of Next-Generation Networks: An LLM-Centric Approach,” *IEEE Network*, vol. 38, no. 5, pp. 29–36, Sept. 2024.

[12] R. P. Orozco and others, “Intent-Based Service Provisioning and Closed-Loop Automation for Cobot Service Migration in a Multi-Stakeholder Environment,” *IEEE Transactions on Network and Service Management*, vol. 22, no. 5, pp. 4192–4205, Oct. 2025.

[13] R. Zhang and others, “Interactive AI With Retrieval-Augmented Generation for Next Generation Networking,” *IEEE Network*, vol. 38, no. 6, pp. 414–424, Nov. 2024.

[14] K. Mehmood and others, “Knowledge-Driven Intent Life-Cycle Management for Cellular Networks,” *IEEE Transactions on Network and Service Management*, vol. 22, no. 5, pp. 4806–4826, Oct. 2025.

[15] Y. Huang and others, “Large Language Models for Networking: Applications, Enabling Techniques, and Challenges,” *IEEE Network*, vol. 39, no. 1, pp. 235–242, Jan. 2025.

[16] A. Angi and others, “LLNet: An Intent-Driven Approach to Instructing Softwarized Network Devices Using a Small Language Model,” *IEEE Transactions on Network and Service Management*, vol. 22, no. 4, pp. 3403–3418, Aug. 2025.

[17] D. H. Hussein and others, “Privacy-Preserving Intelligent Intent-Based Network Slicing for IoT Systems,” *IEEE Internet of Things Journal*, vol. 12, no. 14, pp. 27783–27807, July 2025.

[18] N. Alabbasi and others, “TeleOracle: Fine-Tuned Retrieval-Augmented Generation With Long-Context Support for Networks,” *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13170–13182, May 2026.

[19] A. Maatouk and others, “TeleQnA: A Benchmark Dataset to Assess Large Language Models Telecommunications Knowledge,” *IEEE Network*, p. 1, 2025.

[20] D. Haj Hussein and others, “Toward Intelligent Intent-Based Network Slicing for IoT Systems: Enabling Technologies, Challenges, and Vision,” *IEEE Transactions on Network and Service Management*, vol. 22, no. 4, pp. 3480–3495, Aug. 2025.